arXiv:2603.01499v2 [cs.CR] 30 Mar 2026

TOWARDS PRIVACY-PRESERVING LLM INFERENCE VIA COVARIANT OBFUSCATION (TECHNICAL REPORT)

Yu Lin, Qizhi Zhang, Wenqiang Ruan, Daode Zhang, Jue Hong, Ye Wu *
ByteDance

Hanning Xia, Yunlong Mao, Sheng Zhong
Nanjing University

ABSTRACT

The rapid development of large language models (LLMs) has driven the widespread adoption of cloud-based LLM inference services, while also bringing prominent privacy risks associated with the transmission and processing of private data in remote inference. For privacy-preserving LLM inference technologies to be practically applied in industrial scenarios, three core requirements must be satisfied simultaneously: (1) Accuracy and efficiency losses should be minimized to mitigate degradation in service experience. (2) The inference process can be run on large-scale clusters consist of heterogeneous legacy xPUs. (3) Compatibility with existing LLM infrastructures should be ensured to reuse their engineering optimizations. To the best of our knowledge, none of the existing privacy-preserving LLM inference methods satisfy all the above three constraints while delivering meaningful privacy guarantees. In this paper, we propose `AloePri`, the first privacy-preserving LLM inference method for industrial applications. `AloePri` protects both the input and output data by covariant obfuscation, which jointly transforms data and model parameters to achieve better accuracy and privacy. We carefully design the transformation for each model component (e.g., Attention, FFN) to ensure inference accuracy and data privacy while keeping full compatibility with existing infrastructures of Language Model as a Service (LMaaS). `AloePri` has been integrated into an industrial LMaaS system deployed on a multi-node GPU cluster for the evaluation of mainstream LLMs. The evaluation on Deepseek-V3.1-Terminus model (671B parameters) demonstrates that `AloePri` causes accuracy loss of $0.0\% \sim 3.5\%$ and exhibits efficiency equivalent to that of plaintext inference. Meanwhile, `AloePri` successfully resists state-of-the-art attacks (e.g., internal state inversion), with less than 5% of tokens recovered. To the best of our knowledge, `AloePri` is the first method to exhibit practical applicability to large-scale models in real-world systems.

***K*eywords** Large language model · Privacy-preserving inference · Covariant obfuscation

1 Introduction

In the era of large language models (LLMs), a surge in application demands from clients (e.g. financial companies and hospitals) drives cloud service providers to offer low-cost language model as a service (LMaaS). However, to use LMaaS, clients have to transmit their private data to remote LLM services, bringing severe privacy concerns. While resolving the privacy concerns is important, the following three constraints must be satisfied in an industrial scenario to keep the product competitiveness and a high return on investment (ROI) when designing privacy-enhancement technologies:

1. **Accuracy and Efficiency Constraint.** Privacy protection methods must not incur perceptible accuracy degradation or efficiency loss. A model performance gap larger than a generational leap (e.g., 88.3% for Claude 3.5 Sonnet vs. 85.7% for Claude 3 on MMLU [1]) correlates with perceptible disparities in client-facing

*The authors thank Prof. Cong Wang of the City University of Hong Kong for his valuable suggestions on revising this paper.

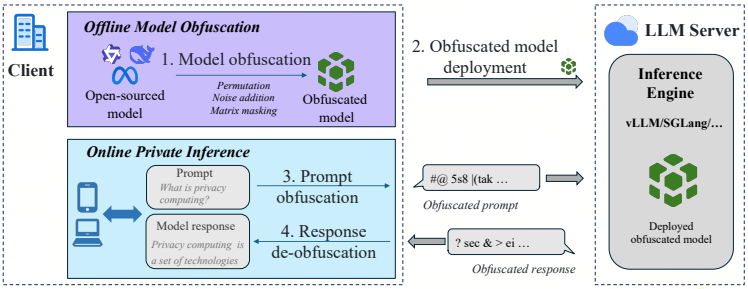


Figure 1: Workflow of `AloePri`. The client locally obfuscates the model and deploys it to the server, and handles prompt obfuscation and response de-obfuscation during online phase.

output quality [2, 3, 4]. End-to-end latency exceeding 100 ms is perceptually discernible to clients [5]. Such accuracy and efficiency impairments would result in an unacceptable decline in product competitiveness.

2. **Hardware Compatibility Constraint.** LLM inference clusters of cloud service providers typically consist of heterogeneous legacy xPUs, e.g., Graphics Processing Units (GPUs), Neural Processing Units (NPUs), and Field Programmable Gate Arrays (FPGAs). Therefore, to keep a high ROI, a privacy protection method must be able to run on heterogeneous legacy xPUs clusters.

3. **Software Compatibility Constraint.** The method should be compatible with existing LLM infrastructures so that their engineering optimizations can be reused. The current inference frameworks (e.g., vLLM, SGLang) have integrated many software optimizations (e.g., KV Cache, P/D disaggregation). Any privacy protection method that is not compatible with these frameworks would require huge engineering efforts to re-implement these software optimizations, leading to a low ROI.

The above constraints severely restrict the applicability of methods based on cryptography or trusted execution environment (TEE). While cryptographic methods [6, 7] can provide rigorous privacy guarantees, their substantial computational overhead and incompatibility with LMaaS infrastructures render them impractical for industrial applications. Meanwhile, a large number of existing computing clusters contain legacy xPUs without hardware-native security guarantees. Therefore, TEE-based methods [8, 9, 10] cannot be widely deployed in such scenarios.

While obfuscation-based methods [11, 12, 13, 14, 15, 16, 17, 18, 19] are promising for privacy-preserving LLM inference because they are efficient and broadly compatible with existing infrastructure, these methods are still far from practical usage.

To quantify privacy guarantees, researchers commonly adopt formal privacy frameworks such as Differential Privacy (DP) [20], Rényi Differential Privacy (RDP) [21], and metric Differential Privacy (mDP) [22]. Mechanisms for realizing these privacy definitions typically employ either additive noise injection (e.g., the Gaussian and Laplace mechanisms) or random replacement strategies (e.g., the exponential mechanism). Building on these mechanism, recent work has proposed obfuscation methods for LLM inference that operate at either the token level [11, 12, 13, 14, 15] or the embedding level [16, 17, 18, 19]. Despite their promise, such methods suffer poor trade-off between privacy and accuracy. As we show in Section 7, several state-of-the-art obfuscation methods [17, 16, 11, 23] suffer an accuracy loss of more than 30% while still being vulnerable to inversion attacks that recover over 50% of tokens.

In this paper, to achieve better privacy, efficiency, and accuracy, we first propose *Covariant Obfuscation*-a novel privacy-preserving LLM inference mechanism that jointly transforms input data and model weights. By transforming model weights to mitigate the impact of input obfuscations during inference, covariant obfuscation can provide a stronger privacy guarantee than data-only obfuscation methods at equal model accuracy. Furthermore, we introduce three composition theorems (sequential, parallel, and summation) to streamline the construction of covariant obfuscation.

Second, following covariant obfuscation, we propose `AloePri`, an `Accurate`, `low`-cost, and `efficient` solution for `Privacy`-preserving LLM inference. As illustrated in Figure 1, model obfuscation is performed offline: the client generates a secret token-level permutation and applies it to token-correlated model weights, including the embedding layer and model head. Additional Gaussian noise and carefully selected invertible matrices are further employed to transform the weights of all model components (e.g., attention, FFN) to hide the secret permutation while preserving model inference accuracy. During online inference, the client obfuscates its prompts before sending them to the server and decodes the returned responses using the secret permutation. Under this covariant obfuscation, the server observes only obfuscated inputs and outputs and thus cannot access the client's private data.

We evaluate AloePri via theoretical analysis and empirical experiments. Theoretical analysis characterizes the computational error introduced by AloePri, establishes upper bounds on its information leakage, and quantifies the corresponding attack success rate. On the other hand, we conduct comprehensive empirical evaluations on mainstream open-source LLMs (including Qwen2.5/Qwen3, Llama3, Qwen3-MoE, Deepseek-R1-Distill-Qwen, and Deepseek-V3.1-Terminus) and test AloePri's resilience against representative attacks [24, 25, 26, 27, 28] on privacy-preserving LLM inference methods. Experimental results show that AloePri significantly mitigates privacy leakage while retaining both model accuracy and inference efficiency. For instance, on Deepseek-V3.1-Terminus, AloePri incurs only $0\% \sim 3.5\%$ accuracy loss, with fewer than 5% of tokens recovered by several state-of-the-art attacks [24, 25, 26, 27, 28]. Furthermore, as its inference pipeline remains nearly identical to plaintext inference, AloePri supports direct integration into mainstream inference frameworks (e.g., vLLM [29] and SGLang [30]) with negligible efficiency overhead and engineering efforts.

We highlight our contributions as follows:

- We propose covariant obfuscation, a novel privacy-preserving LLM inference mechanism. We formalize its compositional properties, enabling the seamless integration into sophisticated LLM architectures. (Section 4)
- Based on the covariant obfuscation mechanism, we propose AloePri that achieves efficiency comparable to plaintext model inference, provides robust privacy protection on both input and output data, and incurs slight accuracy loss. Notably, AloePri does not change the model structure and is compatible with existing LLM infrastructures. (Section 5)
- We introduce a novel Rényi-metric differential privacy definition tailored for analyzing the privacy guarantees of high-dimensional Gaussian noise. Building on this framework, we theoretically prove that AloePri achieves a stronger privacy guarantee than data-only obfuscation by incorporating obfuscation on model weights. (Section 6)
- We conduct comprehensive experiments to validate the effectiveness of AloePri. Experimental results across multiple datasets and LLMs demonstrate that AloePri significantly outperforms state-of-the-art obfuscation methods in both data privacy and model inference accuracy. (Section 7)

2 Related Works

2.1 Methods based on Obfuscation

A series of obfuscation-based methods have been proposed for privacy-preserving LLM inference, primarily via three techniques: token substitution [11, 23, 12, 15], embedding transformation [16, 18, 17, 31, 32, 19], LLM-aided rewriting [14, 33].

Token Substitution. Since prompts are composed of a sequence of tokens, the most intuitive privacy preservation method is to substitute sensitive tokens within the text. SANTEXT [23] and RANTEXT [11] leverage local differential privacy to replace sensitive tokens with alternatives based on token embedding similarity. ProSan [15] quantifies word importance and privacy risk to dynamically balance accuracy and privacy. However, these methods only protect a limited set of sensitive tokens and lack semantic-level protection. Moreover, if substituted tokens correlate with task objectives, they undermine the consistency between input text and task intent.

Embedding Transformation. Some studies obfuscate the token embeddings of input data to achieve semantic-level privacy protection. SGT [17] learns an embedding transformation model via a mutual information-based training loss, converting raw user inputs into noisy embeddings for secure remote LLM inference. DP-Forward [16] designs a differential privacy mechanism to set privacy budgets for high-dimensional embedding perturbation, balancing privacy and inference accuracy. STIP [18] adopts feature-space random permutations to enable efficient private inference in a three-party scenario. These methods require the client and server to transmit data in the form of embeddings, which increases communication overhead. Meanwhile, existing inversion attacks [26, 34, 24, 25] pose a critical threat, as adversaries can invert obfuscated embeddings to recover the original private texts. In addition, following the framework of split inference, some studies deploy part of models at the client side to process and obfuscate data. SnD [32] splits LLMs into local/cloud encoders, such that the client can add noise to hidden states outputted by local encoders and use pre-trained denoiser on cloud encoders' outputs to obtain high-quality inference results. NOIR [19] splits open-source LLMs into a lightweight client-side encoder/decoder and a cloud-hosted middle layer, which are trained jointly to achieve accuracy and privacy. Meanwhile, it introduces the concept of indistinguishability-preserving vocabulary, which requires the client to locally randomize token embeddings and process them with the client-side encoder. While split inference enables the client to better balance accuracy and privacy, it requires the client to participate in a training process and take part of forward computational model layers during inference, which is not compatible with the existing LLM infrastructure.

LLM-aided Rewriting. This type of method leverages LLMs themselves to hide private information in input data. PAPILLON [14] deploys a multi-stage LLM pipeline (local + internet models) to preserve privacy while maintaining high task response quality. EmojiPrompt [33] leverages cloud-hosted LLMs to rewrite private texts with special symbols and emojis. These methods rely on LLMs deployed in a trusted local or third-party environment to perform private text rewriting, and quantifying the privacy guarantees of the rewritten texts remains a significant challenge.

2.2 Methods based on Cryptography and TEE

Cryptographic primitives provide rigorous privacy guarantees, spurring many cryptography-based methods [6, 7, 35]. Gupta et al. [6] combine additive secret sharing (ASS) and function secret sharing for efficient online inference at the cost of massive offline computation and communication overhead. Zhang et al. [7] implement one-round inference via fully homomorphic encryption (FHE). Some studies [35] combine ASS and FHE to balance communication and computation overhead. Li et al. [36] optimize such methods by modifying activation functions. Despite recent advances, these methods remain far from practical. For example, BumbleBee [35] takes about 8 minutes to generate a single token for LLaMA-7B.

TEE-based methods constitute another key route for privacy-preserving LLM inference. PipeLLM [8] and ccAI [37] leverage GPU-TEE to enable confidential LMaaS. By adopting speculative pipelined encryption, PipeLLM limits the additional throughput overhead to less than 20% for LLMs with parameter sizes spanning 13B to 175B. ccAI adds an additional FPGA-based PCI link to secure PCIe packet transmission across diverse xPU types without requiring modifications to applications or drivers. While these methods have promising efficiency, they are either not suitable for heterogeneous legacy xPU clusters or require a huge hardware retrofit cost. Additionally, several approaches [38, 9, 10] have been proposed to leverage CPU-TEE and legacy GPUs for privacy-preserving inference. However, frequent data transfers between CPU-TEE and legacy GPUs render these methods incompatible with key optimizations in modern LLM infrastructure (e.g., KV Cache, P/D disaggregation).

3 Preliminaries

3.1 System Model

We focus on the general scenario of LMaaS. A *client* holding private data aims to access the LLM inference service provided by a cloud *server*. The client is only capable of undertaking a certain level of computational overhead during the offline phase. Meanwhile, during the online inference phase, to avoid compromising inference throughput, they can only handle lightweight computational tasks. The LLM inference service provided shall be compatible with the interfaces of modern LLM frameworks, i.e., the service takes prompts as inputs and outputs inference results via one-round communication. For instance, a software company requests a remote LLM server to use code generation agents.

3.2 Threat Model

Our goal is to protect the private prompt and model response during LLM inference. We assume the server is honest-but-curious: it faithfully provides the inference service but actively seeks to learn the client's private information. The attacker can access the model and observe the service's inputs and outputs. For example, the attacker can try recovering users' private data through the following three methods: 1) **Obfuscation recovery**: the attacker knows the obfuscation mechanism and tries to breach it by exploiting its structural weaknesses [34, 25]. 2) **Training-based inversion**: the attacker can use substantial computational resources to recover private data via dedicated training-based inversion [24, 26]. 3) **Token-frequency exploitation**: the attacker continuously monitors input and output tokens and records their frequencies over time to infer private information [27].

3.3 Notations

We use $\mathcal{V}$ and θ to represent the vocabulary and model weights, respectively. n denotes the number of tokens, and d represents the hidden size. Throughout this paper, S_n is defined as the permutation group over $[1, n]$, O_d is the orthogonal group of $d \times d$ matrices, and id_A denotes the identity transformation on the space A. Noised data is denoted by $\square^\star$, while randomly sampled data is marked with $\hat{\square}$, and $\widetilde{\square}$ denotes obfuscated model weights. We summarize all notations used in this paper in Appendix A.

3.4 Large Language Model

3.4.1 Model Structure

A typical LLM consists of a vocabulary $\mathcal{V}$ with n tokens, as well as five key components: an embedding layer, a model head, L attention layers, L feed-forward network (FFN) layers, and multiple normalization layers.

Vocabulary, Embeddings, and Model Head. The vocabulary $\mathcal{V}$ and corresponding merge rules serve to tokenize textual prompts into a sequence of token indices. The weight matrix of the embedding layer W_e is an $n \times d$-dimensional matrix, and that of the model's head layer W_h is a $d \times n$-dimensional matrix. Each token index in $\mathcal{V}$ corresponds to a d-dimensional vector in W_e and W_h, respectively.

FFN. The structures of FFN vary between two types of LLMs: dense models and Mixture-of-Experts (MoE) models [39]. In dense models, each FFN layer consists of three sets of weights, denoted as $\omega_{\text{ffn}} = (W_{\text{gate}}, W_{\text{up}}, W_{\text{down}})$. Given the hidden states x as input, the forward pass output of the FFN can be calculated as:

$$f_{\text{ffn}}(x, \omega_{\text{ffn}}) = (\text{SiLU}(xW_{\text{gate}}) \odot (xW_{\text{up}}))\, W_{\text{down}}, \tag{1}$$

where SiLU is a commonly used activation function in LLMs, and $\odot$ denotes the Hadamard product. In MoE models, each expert corresponds to an FFN layer with the same structure as that in dense models. Furthermore, an expert router (denoted as W_{router}) is employed to select activated experts during model inference.

Attention Layer. In this paper, we mainly consider four popular attention mechanisms: Multi-head Attention (MHA) [40], Multi-Query Attention (MQA) [41], Grouped-Query Attention (GQA) [42], and Multi-head Latent Attention (MLA) [43]. The weights of the attention layer can be denoted as $\omega_{\text{attn}} = (W_{\text{q}}, W_{\text{k}}, W_{\text{v}}, W_{\text{o}})$. We define the total number of attention heads as m, and the number of heads corresponding to the attention key and value weights as m_{kv}. For example, $\frac{m}{m_{\text{kv}}} > 1$ for GQA, $\frac{m}{m_{\text{kv}}} = 1$ for MHA, and $\frac{m}{m_{\text{kv}}} = m$ for MQA. Taking GQA as an example, the attention weights can be split by the number of heads as: $W_{\text{q}} = \left[W_{\text{q}}^{(1)} \quad \dots \quad W_{\text{q}}^{(m)}\right], W_{\text{k}} = \left[W_{\text{k}}^{(1)} \quad \dots \quad W_{\text{k}}^{(m_{kv})}\right], W_{\text{v}} = \left[W_{\text{v}}^{(1)} \quad \dots \quad W_{\text{v}}^{(m_{kv})}\right], W_{\text{o}} = \left[W_{\text{o}}^{(1)} \quad \dots \quad W_{\text{o}}^{(m)}\right]^T$. For the input hidden states x, the attention output of GQA is computed as

$$f_{\text{attn}}(x, \omega_{\text{attn}}) = \sum_i g\left(\frac{\mathcal{G}(xW_{\text{q}}^{(i)})\mathcal{G}(xW_{\text{k}}^{\eta(i)})^T}{\sqrt{d_{\text{head}}}}\right) xW_{\text{v}}^{\eta(i)} W_{out}^{(i)}. \tag{2}$$

In the above equation, i denotes the head index for W_{q} and W_{o}, while $\eta(i)$ represents the corresponding head index for W_{k} and W_{v}. g denotes the softmax function. $\mathcal{G}$ stands for the positional embedding function, such as Rotary Position Embeddings (RoPE) [44].

Layer Normalization. In this paper, we primarily focus on the Root Mean Square Layer Normalization (RMSNorm), the most commonly used layer normalization method in modern LLMs. RMSNorm utilizes a weight parameter w_{norm} and the root mean square to normalize an input state x with dimension d, which is calculated as: $\text{RMSNorm}(x) = \frac{x \odot w_{\text{norm}}}{\sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2}}$.

3.4.2 Workflow of Auto-regressive Text Generation

During inference, the model accepts a textual prompt as input. Using the vocabulary $\mathcal{V}$, the prompt is first tokenized into a sequence of l tokens $\mathcal{T} = \{tok_1, \dots, tok_l\}$, which are further converted to a token index sequence $x = \{x_i\}_{i=1}^{l}$. Let θ denote the set of all model weights. The model's forward computation can be formulated as $f(x, \theta) = y$, which yields a token index $y \in \mathbb{Z}_n$. Specifically, the l token indices first retrieve the corresponding l embeddings via W_e. These embeddings are then fed into the attention and FFN layers to extract semantic information. Finally, the extracted features are used to compute logits with W_h, from which the index of the next generated token y is sampled. To enable auto-regressive generation, the generated token index y is appended to the original token index sequence x to serve as the input for the subsequent inference step.

4 Covariant Obfuscation

In this section, we first present the definition of covariant obfuscation. We then introduce three composition theorems that allow us to separately design covariant obfuscation for each component of the LLM, and subsequently combine these individual covariant obfuscations as a covariant obfuscation for the whole model.

4.1 Definition and Property

4.1.1 Covariant Obfuscation on Inference Function

输入混淆、模型变换、标签混淆、标签去混淆、混淆上推理

We consider an inference function $f : X \times \Theta \to Y$, where X is the data space, Θ is the parameter space, and Y is the prediction space, and $\tilde{X}, \tilde{Y}, \tilde{\Theta}$ are their obfuscated spaces respectively. A covariant obfuscation C with obfuscation error (a constant) e_C for the function f can be represented as the following quintuple $(\phi_X, \phi_\Theta, \phi_Y, \psi_Y, \tilde{f})$: (1) Data obfuscation: $\phi_X : X \to \tilde{X}$. (2) Model transformation: $\phi_\Theta : \Theta \to \tilde{\Theta}$. (3) Label obfuscation: $\phi_Y : Y \to \tilde{Y}$. (4) Label de-obfuscation $\psi_Y : \tilde{Y} \to Y$. (5) Inference in the obfuscated space: $\tilde{f} : \tilde{X} \times \tilde{\Theta} \to \tilde{Y}$. They satisfy the following conditions:

A. Commutation condition: The operations of the obfuscation and the inference function shall be commutative. That is, $\mathbb{E}[d(\tilde{y}, \phi_Y \circ f(x, \theta))] \leq e_C$, where $\tilde{y} = \tilde{f}(\phi_X(x), \phi_\Theta(\theta))$, and $d(\cdot, \cdot)$ is a distance function on $\tilde{Y}$ (e.g. Euclid Distance), indicating that the error incurred by the obfuscation is bounded by e_C.

先混淆再推理与先推理再混淆结果相似

$$\begin{array}{ccc} X \times \Theta & \xrightarrow{f} & Y \\ {\scriptstyle (\phi_X, \phi_\Theta)}\downarrow & & \downarrow{\scriptstyle \phi_Y} \\ \tilde{X} \times \tilde{\Theta} & \xrightarrow{\tilde{f}} & \tilde{Y} \end{array}$$

B. De-obfuscation condition: $\psi_Y \circ \phi_Y = \mathrm{id}_Y$, where id_Y denotes the identity transformation on Y.

Notably, when $\tilde{X} = X, \tilde{\Theta} = \Theta, \tilde{Y} = Y, \phi_\Theta = \mathrm{id}_\Theta$, and $\phi_Y = \psi_Y = \mathrm{id}_Y$, the covariant obfuscation degenerates to a data-only obfuscation given by $\phi_X : X \to X$.

乱码输出必须能解回原文

4.2 Composition Theorems

As LLM includes complicated structures, we formalize the compositional theorems (sequential, parallel, and summation) of covariant obfuscation to enable the design of covariant obfuscation for basic model components. The proofs of the theorems are presented in Appendix B.

串联：前一层输出可被后一层识别，误差被逐层放大

4.2.1 Sequential Composition

We show that two sequentially connected covariant obfuscations designed for two LLM adjacent components (e.g., W_v and W_o) still constitute a covariant obfuscation, whose obfuscation error is controlled by the obfuscation errors of two original covariant obfuscations.

Theorem 1 (Sequential Composition Theorem $f \circ g$)**.** *Assume that a covariant obfuscation C_1 for $f : X \times \Theta \to Y$, and a covariant obfuscation C_2 for $g : Y \times \Xi \to Z$ satisfying the boundary condition: $\phi_{Y|C_1} = \phi_{Y|C_2}$ (denoted as ϕ_Y uniformly below). Then, $C_2 \circ C_1 := (\phi_X, (\phi_\Theta, \phi_\Xi), \phi_Z, \psi_Z, \tilde{h})$ is a covariant obfuscation for $h : X \times (\Theta \times \Xi) \to Z$ where $h(x, (\theta, \xi)) := g(f(x, \theta), \xi)$.*

Furthermore, let e_{C_1} and e_{C_2} be the obfuscation errors of C_1 and C_2, respectively. For all $\tilde{y}_1, \tilde{y}_2 \in \tilde{Y}$ and $\tilde{\xi} \in \tilde{\Xi}$, if $\tilde{g}(\tilde{y}, \tilde{\xi})$ satisfies the Lipschitz condition:

$$d(\tilde{g}(\tilde{y}_1, \tilde{\xi}), \tilde{g}(\tilde{y}_2, \tilde{\xi})) \leq M_g \cdot d(\tilde{y}_1, \tilde{y}_2),$$

where M_g is the Lipschitz constant for $\tilde{g}$. The obfuscation error of the series composition satisfies:

$$e_{C_2 \circ C_1} \leq M_g \cdot e_{C_1} + e_{C_2}.$$

并联：输入混淆相同，误差相加

4.2.2 Parallel Composition

Some LLM components are connected in parallel, e.g., attention queries and keys. When the covariant obfuscations of these components are connected in parallel, they also constitute a covariant obfuscation.

Theorem 2 (Parallel Composition Theorem $f||g$)**.** *Assume that a covariant obfuscation C_1 for $f : X \times \Theta \to Y$ and a covariant obfuscation C_2 for $g : X \times \Xi \to Z$ satisfying the boundary condition: $\phi_{X_{|C_1}} = \phi_{X_{|C_2}}$ (denoted as ϕ_X uniformly below). Then, $C_1||C_2 := (\phi_X, (\phi_\Theta, \phi_\Xi), (\phi_Y, \phi_Z), (\psi_Y, \psi_Z), \tilde{h})$ is a covariant obfuscation for $h : X \times (\Theta \times \Xi) \to Y \times Z$, where $h(x, (\theta, \xi)) := (f(x, \theta), g(x, \xi))$.*

Furthermore, let e_{C_1} and e_{C_2} be the obfuscation errors of C_1 and C_2, if the distance function $d_{Y \times Z}$ on $Y \times Z$ satisfies the control condition with respect to the distance functions d_Y on Y and d_Z on Z:

$$d_{Y \times Z}((y_1, z_1), (y_2, z_2)) \leq d_Y(y_1, y_2) + d_Z(z_1, z_2),$$

then $e_{C_1||C_2} \leq e_{C_1} + e_{C_2}$.

加法：输入、输出混淆均相同、在同一个空间，误差相加

4.2.3 Summation Composition

The covariant obfuscations for bypass computations, e.g., residual connections, can also be composed.

Theorem 3 (Summation Composition Theorem $f+g$)**.** *Assume that a covariant obfuscation C_1 for $f : X \times \Theta \to Y$ and a covariant obfuscation C_2 for $g : X \times \Xi \to Y$ satisfying the boundary condition: $\phi_{X_{|C_1}} = \phi_{X_{|C_2}}, \phi_{Y_{|C_1}} = \phi_{Y_{|C_2}}, \psi_{Y_{|C_1}} = \psi_{X_{|C_2}}$ (denoted as ϕ_X, ϕ_Y, ψ_Y uniformly below), where Y and $\tilde{Y}$ are Abelian groups, and $\phi_Y : Y \to \tilde{Y}$ is a group homomorphism. Then, $C_1 + C_2 := (\phi_X, (\phi_\Theta, \phi_\Xi), \phi_Y, \psi_Y, \tilde{h})$ is a covariant obfuscation for for $h : X \times (\Theta \times \Xi) \to Y$ where $h(x, (\theta, \xi)) := f(x, \theta) + g(x, \xi)$ such that $\tilde{h}(x, (\theta, \xi)) := \tilde{f}(x, \theta) + \tilde{g}(x, \xi)$.*

Furthermore, let e_{C_1} and e_{C_2} be the obfuscation errors of C_1 and C_2, if the distance function on Y satisfies translation invariance, i.e.,

$$d(x, y) = d(x + z, y + z),$$

then $e_{C_1+C_2} \leq e_{C_1} + e_{C_2}$.

5 AloePri

In this section, we first present an overview of `AloePri`, then describe the covariant obfuscation design, as well as the analysis on accuracy and information leakage in detail.

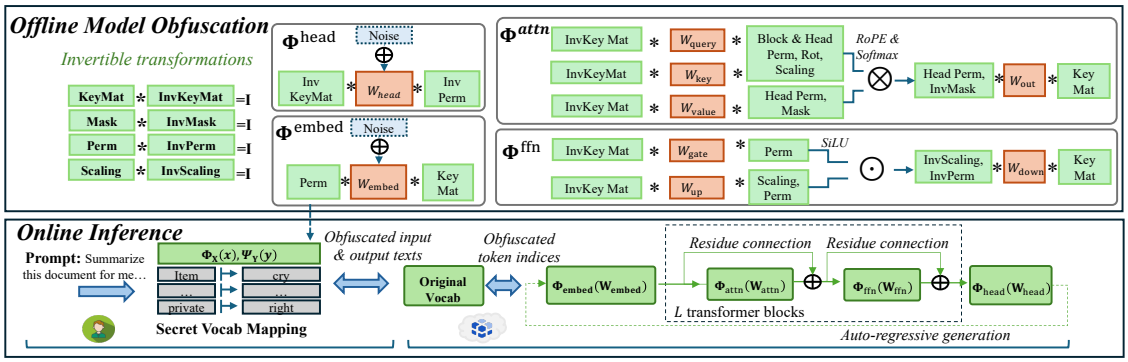


Figure 2: Overview of `AloePri`. In the offline model obfuscation process, token-level permutation and linear transformations are employed to construct the obfuscations ϕ^{embed}, ϕ^{head}, ϕ^{attn}, and ϕ^{ffn}. In the online inference process, a secret vocabulary mapping associated with the permutation is used to construct the data obfuscation ϕ_X and de-obfuscation ψ_Y.

5.1 Overview

As shown in Figure 2, based on the covariant obfuscation mechanism, `AloePri` includes both data obfuscation ϕ_X and model obfuscation ϕ_Θ mechanisms for an LLM inference function $f : \mathbb{Z}_n^l \times \Theta \to \mathbb{Z}_n$. In an offline model obfuscation process, the client generates a secret permutation τ, which is used in $\phi_X = \tau(x), \phi_Y(y) = \tau(y)$ to permute input and output token indices x, y. The permutation τ is also used to shuffle the embedding and model head at the token level. To prevent the attacker from recovering τ by comparing plaintext and obfuscated weights of the embedding and model head, a series of sub-obfuscations, including ϕ^{embed}, ϕ^{head}, ϕ^{attn}, and ϕ^{ffn}, are applied to all model weights. These sub-obfuscations also protect the internal states generated during model inference. Based on the composition theorems in Section 4.2, we combine all the sub-obfuscations into ϕ_Θ, which together with ϕ_X, ϕ_Y to form the covariant obfuscation of f for the entire model inference process. During the online inference process, the client locally obfuscates its text data at the token level based on τ before sending them to the server. According to the definition of covariant obfuscation, `AloePri` can preserve model accuracy when performing inference on obfuscated data using obfuscated model weights.

Remark. `AloePri` leverages lightweight token-level permutations to protect tokens, and uses invertible transformations and noise injection to conceal the permutation itself. Following previous studies [45, 46] on practical privacy-enhancement technologies, `AloePri` aims to provide adjustable security for constrained attackers in real-world scenarios, rather than offering ideal-world security guarantees against worst-case attackers. To estimate the security of `AloePri`, we extend the mDP [47] and RDP [21] to derive a novel Rényi-metric Differential Privacy, which is more

suitable for analyzing high-dimensional Gaussian noise, in Section 6. We also empirically validate the privacy guarantee against constrained attackers via a series of experiments in Section 7.

5.2 Offline Model Obfuscation

In offline model obfuscation, token-level permutation is applied to weights of the embedding layer and model head, which enables the client to obfuscate texts at the token level and supports auto-regressive text generation tasks. Meanwhile, by introducing *two-side* (left-multiplication and right-multiplication) transformations for weight matrices, the token-level permutation is kept secret from the server. We first present an algorithm to generate (inverse) key matrix used to construct obfuscations. Then we present concrete construction of ϕ^{embed}, ϕ^{head}, ϕ^{attn}, ϕ^{ffn} for popular LLM structures.

5.2.1 Key Matrix Generation

During LLM inference, input data is processed layer-by-layer between adjacent layers with residual connections. `AloePri` utilizes a set of key matrices and their inverses to perform two-side transformations on the weights of adjacent layers. Therefore, without knowledge of the key matrices (or their inverses), the attacker cannot recover the secret token-level permutation imposed on the embedding layer and model head. Meanwhile, each key matrix can be canceled out by any of its inverses, ensuring the correctness of forward computation.

Algorithm 1 shows the generation of key matrices and their inverses. The algorithm takes the hidden size d of the model, expansion size h, and matrix coefficient λ as inputs to initialize a set of base matrices. Then, `KeyMatGen` and `InvKeyMatGen` leverage the base matrices to generate random key matrices $\{\hat{P}\}$ and inverse key matrices $\{\hat{Q}\}$. With the expanded dimension controlled by h, we can generate an infinite number of matrices $\hat{P}$ and $\hat{Q}$ that satisfy $\hat{P} \cdot \hat{Q} = I$ using `KeyMatGen` and `InvKeyMatGen`. Key and inverse key matrices are used to obfuscate model weights, including the embedding, attention, FFN, and model head layers. Since most LLMs store model weights in half-precision, an extra parameter λ is employed to regulate the matrix norm, avoiding significantly altering the magnitude of obfuscated weights.

Algorithm 1 Key Matrix Generation

Parameters: hidden size d, expansion size h, coefficient λ.

function INIT(d, h, λ)
  Uniformly sample U from orthogonal group O_d.
  Sample $V \sim \mathcal{N}(0, \frac{1}{d})^{d \times d}$, set $B = U + \lambda V$, compute B^{-1}.
  Sample $E_1 \sim \mathcal{N}(0, \frac{1}{d})^{d \times h/2}$, $E_2 \sim \mathcal{N}(0, \frac{1}{d})^{h/2 \times h}$, set $E = E_1 E_2$.
  Sample $F_1 \sim \mathcal{N}(0, \frac{1}{d})^{h \times h/2}$, $F_2 \sim \mathcal{N}(0, \frac{1}{d})^{h/2 \times d}$, set $F = F_1 F_2$.
  Sample a orthogonal matrix $Z \in O_{d+2h}$ uniformly.
  return (B, B^{-1}, E, F, Z).
end function

function KEYMATGEN(B, E, F, Z)
  Construct $C \in \mathbb{R}^{d \times h}$ with columns sampled from $\text{null}(F^T)$.
  return $\hat{P} = [\, B \; C \; E \,] Z$.
end function

function INVKEYMATGEN(B^{-1}, E, F, Z)
  Construct $D \in \mathbb{R}^{h \times d}$ with rows sampled from $\text{null}(E)$.
  return $\hat{Q} = Z^T [\, B^{-1} \; F \; D \,]^T$.
end function

5.2.2 Embedding and Model Head Obfuscation

Since the embedding layer and model head are directly associated with the input and output of the model, we also need to take into account the design of the data obfuscation ϕ_X when constructing ϕ^{embed} and ϕ^{head}. `AloePri` encompasses three techniques to construct ϕ^{embed} and ϕ^{head}: noise addition, permutation, and (inverse) key matrix transformations.

Noise Addition. The client samples noise matrices $\mathcal{E}_{\text{embed}} \sim \mathcal{N}(0, \sigma_e^2 I_n \otimes I_d)$ and $\mathcal{E}_{\text{head}} \sim \mathcal{N}(0, \sigma_h^2 I_d \otimes I_n)$, where σ_e, σ_h are the standard variation of W_e, W_h. The noisy weights of embedding and model head can be expressed as $W^{\star}_{\text{embed}} = W_e + \alpha_{\text{e}} \cdot \mathcal{E}_{\text{embed}}$ and $W^{\star}_{\text{head}} = W_h + \alpha_{\text{h}} \cdot \mathcal{E}_{\text{head}}$, where α_{e} and α_{h} are noise parameters.

Permutation and Key Matrix Transformation. The client further employs token-level permutation to construct $\phi^{\text{embed}}, \phi^{\text{head}}$. This permutation keeps the support of auto-regressive generation. Meanwhile, the permutation also

correlates to the construction of ϕ_X and ϕ_Y so that the attacker cannot recover private input and output data without knowing the permutation. Specifically, the client samples a permutation $\tau \sim S_n$. Let Π be the permutation matrix corresponding to τ. This permutation matrix is applied to $W^{\star}_{\text{embed}}, W^{\star}_{\text{head}}$. Simultaneously, a key matrix $\hat{P}_{\text{embed}}$ and a inverse key matrix $\hat{Q}_{\text{head}}$ are generated with Algorithm 1 and used to obfuscate $W^{\star}_{\text{embed}}, W^{\star}_{\text{head}}$. Finally, the obfuscation of embedding and model head are formalized as: $\widetilde{W}_{\text{embed}} = \Pi W^{\star}_{\text{embed}} \hat{P}_{\text{embed}}$, $\widetilde{W}_{\text{head}} = \hat{Q}_{\text{head}} W^{\star}_{\text{head}} \Pi^T$. Besides the obfuscation of embedding and model head, the client uses the permutation τ to generate a secret token mapping $\mathcal{Z} = \{\mathcal{V}[i] : \mathcal{V}[\tau[i]]\}$, which is used to obfuscate data during online inference.

5.2.3 Attention Obfuscation

As described in Eq. 2, an attention layer processes data using each attention head independently, and then aggregates the outputs across all heads. Therefore, we obfuscate the attention weights with the following two types of transformation.

Intra-head Transformation. We use Algorithm 2 to obfuscate the weights of a group of attention heads $(W_{\text{q}}^{(i)}, W_{\text{k}}^{\eta(i)}, W_{\text{v}}^{\eta(i)}, W_{\text{o}}^{(i)})$. Key matrices $\hat{Q}_{\text{q}}, \hat{Q}_{\text{k}}, \hat{Q}_{\text{v}}, \hat{P}_{\text{o}}$ (sampled via Algorithm 1) are applied to transform the weights. Random invertible matrices $\hat{U}_{vo}$ are used for value/output weights to preserve computation correctness. Considering RoPE, 2-dimensional rotary matrices $\hat{R}_{qk}$ and scaling matrices $\hat{H}_{qk}$ are introduced to transform query/key weights. Besides, we find that simultaneously shuffling the RoPE's 2×2 blocks of query/key weights within a limited window exerts minimal impact on model accuracy, particularly for blocks with larger indices. Therefore, we perform block-wise permutation within a dynamic window to boost the obfuscation level.

Algorithm 2 Intra-head attention obfuscation

Input: Attention weights $(W_{\text{q}}^{(i)}, W_{\text{k}}^{\eta(i)}, W_{\text{v}}^{\eta(i)}, W_{\text{o}}^{(i)})$, maximum window size β, window sampling parameter γ, RoPE parameter ζ, and block number m_{blocks}.
Output: Obfuscated weights $(\widetilde{W}_{\text{q}}^{(i)}, \widetilde{W}_{\text{k}}^{\eta(i)}, \widetilde{W}_{\text{v}}^{\eta(i)}, \widetilde{W}_{\text{o}}^{(i)})$.

Uniformly sample ρ_i from $(0, 2\pi]$ and generate rotary matrix $\hat{R}_{\text{qk}} = \text{Diag}(\{R_i\}_{i \le \frac{d_{head}}{2}})$, where $R_i = \begin{pmatrix} \cos\rho_i & -\sin\rho_i \\ \sin\rho_i & \cos\rho_i \end{pmatrix}$.
Sample $s_i \in \mathbb{R}$ and define $\hat{H}_{\text{qk}} = \text{Diag}(\{s_1 I_2, \ldots, s_{d_{\text{head}}/2} I_2\})$.
Sample $\hat{Z}_{\text{block}} \leftarrow BlockPerm(\beta, \gamma, \zeta, m_{\text{blocks}})$.
Sample $\hat{U}_{\text{vo}} \sim \mathcal{N}(0, \frac{1}{d_{\text{head}}} I_{d_{\text{head}}} \otimes I_{d_{\text{head}}})$.
Sample $\hat{Q}_{\text{q}}, \hat{Q}_{\text{k}}, \hat{Q}_{\text{v}}, \hat{P}_{\text{o}}$ by Algorithm 1.
Set $\widetilde{W}_{\text{k}}^{\eta(i)} = \hat{Q}_{\text{k}} W_{\text{k}}^{\eta(i)} \hat{R}_{\text{qk}} \hat{H}_{\text{qk}}^{-1} \hat{Z}_{\text{block}}^T$, $\widetilde{W}_{\text{v}}^{\eta(i)} = \hat{Q}_{\text{v}} W_{\text{v}}^{\eta(i)} \hat{U}_{\text{vo}}$.
Set $\widetilde{W}_{\text{q}}^{(i)} = \hat{Q}_{\text{q}} W_{\text{q}}^{(i)} \hat{R}_{\text{qk}} \hat{H}_{\text{qk}} \hat{Z}_{\text{block}}$, $\widetilde{W}_{\text{o}}^{(i)} = \hat{U}_{\text{vo}}^{-1} W_{\text{o}}^{(i)} \hat{P}_{\text{o}}$.
return $(\widetilde{W}_{\text{q}}^{(i)}, \widetilde{W}_{\text{k}}^{\eta(i)}, \widetilde{W}_{\text{v}}^{\eta(i)}, \widetilde{W}_{\text{o}}^{(i)})$

function BLOCKPERM($\beta, \gamma, \zeta, m_{\text{blocks}}$)
    Initialize a list $\mathcal{U} = \{\}$ and set block index $t = 1$.
    Compute $\{\zeta_i = \zeta^{-2(i-1)/m_{\text{blocks}}} | 1 \le i \le m_{\text{blocks}}\}$.
    while $t < m_{\text{blocks}}$ **do**
        Set $c = \min(\beta, m_{\text{blocks}} - t)$.
        Compute: $u = softmax(\{\zeta_{t+i} - \zeta_t | 1 \le i \le c\})$.
        Sample window size w from $[1, c]$ with probabilities u.
        Uniformly sample a permutation matrix $Z \in S_w$ and append it to $\mathcal{U}$.
    end while
    Concatenate the matrices into a block diagonal form: $\hat{Z}_{\text{block}} = \text{BlockDiag}(\mathcal{U})$.
    Return $\hat{Z}_{\text{block}}$.
end function

Inter-head Permutation. The attention weights are further obfuscated via attention head permutation, where we sample two random permutations: $\tau_{\text{kv}} \sim S_{m_{\text{kv}}}$ and $\tau_{\text{group}} \sim S_{m/m_{\text{kv}}}$. τ_{kv} shuffles key and value weights at the individual attention head level, query and output weights at the grouped-head level. Meanwhile, τ_{group} shuffle query and output weights within each group, which disrupts inter-head correlations while preserving the model's aggregation capability. The above obfuscation method can be directly applied to MHA, MQA, and GQA. As for MLA, low-rank matrices are employed for query and key weights, with the integration of decoupled RoPE. Therefore, we obfuscate the low-rank weights in MLA using another set of invertible transformations.

5.2.4 FFN Obfuscation

To ensure the correctness of non-linear operations in FFN, we mainly used scaling and permutation transformation to obfuscate $\omega_{\text{ffn}} = \{W_{\text{gate}}, W_{\text{up}}, W_{\text{down}}\}$. Specifically, the obfuscated FFN weights are computed by: $\widetilde{W}_{\text{gate}} = \hat{Q}_{\text{gate}} W_{\text{gate}} \hat{Z}_{\text{ffn}}, \widetilde{W}_{\text{up}} = \hat{Q}_{\text{up}} W_{\text{up}} \hat{H}_{\text{ffn}} \hat{Z}_{\text{ffn}}, \widetilde{W}_{\text{down}} = \hat{Z}_{\text{ffn}}^{-1} \hat{H}_{\text{ffn}}^{-1} W_{\text{down}} \hat{P}_{\text{down}}$, where $\hat{Z}_{\text{ffn}}$ is a permutation matrix uniformly sampled from $S_{d_{\text{ffn}}}$, and $\hat{H}_{\text{ffn}}$ is a randomly sampled scaling matrix.

For MoE models, an additional router weight W_{router} is incorporated to select experts. To obfuscate W_{router}, we first normalize the weight vector corresponding to every expert to get W'_{router}. Meanwhile, we sample a $m_{\text{exp}} \times m_{\text{exp}}$-dimensional permutation matrix $\hat{Z}_{\text{router}}$ to generate $\widetilde{W}_{\text{router}} = \hat{Q}_{\text{router}} W'_{\text{router}} \hat{Z}_{\text{router}}$. The order of experts is shuffled according to $\hat{Z}_{\text{router}}$, ensuring that the experts can be selected correctly.

5.2.5 Layer Normalization Transformation

Let $W_{\text{norm}} = \text{Diag}(w_{\text{norm}})$ denote the diagonal matrix corresponding to the RMSNorm weights. Assuming that the input data x of any normalization layer follows a Gaussian distribution, we use $\kappa = \mathbb{E}[\frac{||x\hat{P}||}{||x||}]$ as the coefficient for the obfuscated normalization layer to adjust for the bias induced by the $\hat{P}$ transformation. We then fuse an RMSNorm layer with weights $\widetilde{w}_{\text{norm}} = \mathbf{1} \cdot \kappa$ and a linear layer with weights W_{norm} to replace the plaintext RMSNorm layer. The weights of the linear layer W_{norm} can be merged into the layer adjacent to the RMSNorm layer before applying weight obfuscation.

5.3 Online Inference

In the online inference phase, the client leverages the secret token mapping $\mathcal{Z}$ to obfuscate input prompts. Specifically, the client first tokenizes a prompt locally into l tokens $\mathcal{T} = \{tok_1, \cdots, tok_l\}$, then encodes each token tok_i into its obfuscated counterpart $\widetilde{tok}_i = \mathcal{Z}[tok_i]$. Once all obfuscated tokens are generated, the client detokenizes them to construct an obfuscated prompt $\widetilde{\mathcal{T}}$ and sends it to the server. The server tokenizes $\widetilde{\mathcal{T}}$ to produce obfuscated token indices $\{\widetilde{x}_i\}_{i \leq l}$ as the input of the obfuscated model. The model then outputs an obfuscated response to the client. Finally, the client utilizes $\mathcal{Z}$ to decode the obfuscated response back to plaintext.

Remark. Leveraging covariant obfuscation, which is capable of performing obfuscation in the data space, `AloePri` is compatible with data obfuscation techniques operating on the token space for LLM inference. For example, similar to RANTEXT [11], SANTEXT [23], and CUSTEXT [48], `AloePri` can perform token perturbation based on the similarity of token embeddings before applying secret token mapping during online inference.

5.4 Accuracy Analysis

We first analyze the obfuscation-induced bias of each LLM component individually, then leverage the composition theorems for covariant obfuscation to show that the inference accuracy of LLMs can be preserved. We adopt the symbol $\approx_{e_C}$ to denote the approximation relation with the obfuscation error e_C as a constant upper bound.

Embedding and Model Head. Let x denote an input token index, and $f_{\text{embed}}, \widetilde{f}_{\text{embed}}$ the embedding functions in the plaintext and obfuscated spaces, respectively. The covariant obfuscation of the embedding layer is formalized as: $\phi_X^{\text{embed}}(x) = \tau(x), \phi_\Theta^{\text{embed}}(W_e) = \Pi W_{\text{embed}}^\star \hat{P}_{\text{embed}}, \phi_Y^{\text{embed}}(y) = y\hat{P}_{\text{embed}}, \psi_Y^{\text{embed}}(\widetilde{y}) = \widetilde{y}\hat{Q}$. Accordingly, $\widetilde{f}_{\text{embed}}(\phi_X^{\text{embed}}(x), \phi_\Theta^{\text{embed}}(W_e)) \approx_{e_C^{\text{embed}}} \phi_Y^{\text{embed}} \circ f_{\text{embed}}(x, W_e)$, where e_C^{embed} is the obfuscation error of this covariant obfuscation. Similarly, we derive $\widetilde{f}_{\text{head}}(\phi_X^{\text{head}}(x), \phi_\Theta^{\text{head}}(W_h)) \approx_{e_C^{\text{head}}} \phi_Y^{\text{head}} \circ f_{\text{head}}(x, W_h)$, with e_C^{head} as the corresponding obfuscation error.

Attention. We denote $\phi_X^{\text{attn}}(x) = x\hat{P}$ as the obfuscated input to the attention layer $\widetilde{\omega}_{\text{attn}}$. The attention score for the i-th head is computed as: $\mathcal{G}(\widetilde{x}\widetilde{W}_{\text{q}}^{(i)})\mathcal{G}(\widetilde{x}\widetilde{W}_{\text{k}}^{\eta(i)})^T \approx_{e_C^{\text{attn}}} \mathcal{G}(xW_{\text{q}}^{(i')})\mathcal{G}(xW_{\text{k}}^{\eta(i')})^T$, where i' denotes the permuted index corresponding to the i-th head, and e_C^{attn} is the obfuscation error induced by block permutation (parameterized by β, γ). Meanwhile, the attention value satisfies: $\widetilde{x}\widetilde{W}_{\text{v}}^{\eta(i)}\widetilde{W}_{\text{o}}^{(i)} = xW_{\text{v}}^{\eta(i')}W_{\text{o}}^{(i')}\hat{P}_{\text{o}}$. Thus, by integrating attention scores and values, we construct the covariant obfuscation as follows: $\phi_X^{\text{attn}}(x) = x\hat{P}$, $\phi_\Theta^{\text{attn}}(\omega_{\text{attn}}) = \widetilde{\omega}_{\text{attn}}$, $\phi_Y^{\text{attn}}(y) = y\hat{P}_{\text{o}}$, $\psi_Y^{\text{attn}}(\widetilde{y}) = \widetilde{y}\hat{Q}$, with $\widetilde{f}_{\text{attn}}$ denoting the attention function over obfuscated spaces. This obfuscation satisfies: $\widetilde{f}_{\text{attn}}(\phi_X^{\text{attn}}(x), \phi_\Theta^{\text{attn}}(\omega_{\text{attn}})) \approx_{e_C^{\text{attn}}} \phi_Y^{\text{attn}} \circ f_{\text{attn}}(x, \omega_{\text{attn}})$, where $f_{\text{attn}}(x, \omega_{\text{attn}})\hat{P}_{\text{o}} = \phi_Y^{\text{attn}} \circ f_{\text{attn}}(x, \omega_{\text{attn}})$.

FFN. Taking an $\widetilde{x} = x\hat{P}$ as input, the forward computation of the obfuscated FFN layer holds that: $\widetilde{f}_{\text{ffn}}(\widetilde{x}, \widetilde{\omega}_{\text{ffn}}) = (\text{SiLU}(xW_{\text{gate}}) \odot (xW_{\text{up}}))\, W_{\text{down}}\hat{P}_{\text{down}} = f_{\text{ffn}}(x, \omega_{\text{ffn}})\hat{P}_{\text{down}} = \phi_Y^{\text{ffn}} \circ f_{\text{ffn}}(x, \omega_{\text{ffn}})$, where $\phi_Y^{\text{ffn}}(y) = y\hat{P}_{\text{down}}$ and $\widetilde{f}_{\text{ffn}}$ is the FFN function defined over the obfuscated spaces.

Layer Normalization. Let $W_{\text{norm}} = \text{Diag}(w_{\text{norm}})$, we construct the covariant obfuscation as: $\phi_X^{\text{norm}}(x) = x\hat{P}$, $\phi_\Theta^{\text{norm}}(w_{\text{norm}}) = \widetilde{w}_{\text{norm}}$, $\phi_Y^{\text{norm}}(y) = yW_{\text{norm}}^{-1}\hat{P}$, $\psi_Y^{\text{norm}}(\widetilde{y}) = \widetilde{y}\hat{Q}W_{\text{norm}}$, and $\widetilde{f}_{\text{RMSNorm}}$ denotes the RMSNorm function defined over the obfuscated spaces. We have $\widetilde{f}_{\text{RMSNorm}}(\widetilde{x}, \widetilde{w}_{\text{norm}}) = \frac{\boldsymbol{x}\hat{P}}{\sqrt{\frac{1}{d+2h}\sum_i^{d+2h}\widetilde{x}_i^2}}\kappa I \approx_{e_C^{\text{norm}}} \frac{\boldsymbol{x}\hat{P}}{\sqrt{\frac{1}{d}\sum_i^d x_i^2}} = \phi_Y^{\text{norm}} \circ f_{\text{RMSNorm}}(x, w_{\text{norm}})$, where e_C^{norm} is the obfuscation error of the covariant obfuscation.

Putting Together. The above components are sequentially connected with residue connections in typical LLM structures, and the error of each component is relatively small. Since the components satisfy the bound conditions of Theorem 1 and Theorem 3, we can integrate these components to construct the covariant obfuscation as: $\phi_X(x) = \tau(x)$, $\phi_\Theta(\theta) = (\widetilde{W}_{\text{embed}}, \widetilde{W}_{\text{head}}, \{\widetilde{\omega}_{\text{attn}}, \widetilde{\omega}_{\text{ffn}}, \widetilde{w}_{\text{norm}}\})$, $\phi_Y(y) = \tau(y)$, $\psi_Y(\widetilde{y}) = \tau^{-1}(\widetilde{y})$, and $\widetilde{f}$ is the LLM inference function for obfuscated spaces. It holds that $\widetilde{f}_\Theta(\phi_X(x), \phi_\Theta(\theta)) \approx_{e_C^{\texttt{AloePri}}} \phi_Y \circ f_\Theta(x, \theta)$, where $e_C^{\texttt{AloePri}}$ is the obfuscation error of `AloePri`.

As $e_C^{\texttt{AloePri}}$ is related to the structure of LLM, we take a typical dense model structure like Qwen2 [49] as an example. The obfuscation error of i-th decoder layer can be derived as

$$e_{C_i}^{\text{decoder}} \leq (M_i^{\text{norm}}(M_i^{\text{attn}} e_{C_i}^{\text{norm}} + e_{C_i}^{\text{attn}}) + e_{C_i}^{\text{norm}}) M_i^{\text{norm}} M_i^{\text{FFN}},$$

where M^{norm}, M_i^{attn}, M_i^{FFN} are the Lipschitz constants of i-th RMSNorm, attention, FFN layers with consideration of residue connection. Let M^{head} and M_i^{decoder} denote the Lipschitz constants of model head and i-th decoder layer, respectively. It holds that $e_C^{\texttt{AloePri}} \leq \mathcal{M}_0 e_C^{\text{embed}} + \sum_{i=1}^L \mathcal{M}_i e_{C_i}^{\text{decoder}} + e_C^{\text{head}}$, where $\mathcal{M}_i = M^{\text{head}} \prod_{j=i+1}^L M_i^{\text{decoder}}$.

6 Security Analysis

In this section, we introduce Rényi-metric Differential Privacy (RmDP). Building on RmDP, we first analyze the privacy budget of the secret permutation τ associated with data obfuscation. We then compare the privacy guarantees of sensitive tokens provided by `AloePri` and other data-only obfuscation methods.

6.1 Rényi-metric Differential Privacy

DP is a natural framework for analyzing privacy protection methods, but it is not directly applicable to our setting. On the one hand, the privacy goal in LLM inference is to protect clients' private texts, which differs fundamentally from the notion of adjacent datasets in DP. On the other hand, the high-dimensional Gaussian noise commonly used in this setting is difficult to characterize under standard DP. To address these two issues in a unified way, we introduce RmDP in Definition 1.

Definition 1 (Rényi-metric Differential Privacy)**.** *Let $(\mathcal{X}, d)$ be a metric space. A randomized mechanism $\mathcal{M}\colon \mathcal{X} \to \mathcal{Y}$ satisfies* (α, ϵ, d)-RmDP *if, for any $x, x' \in \mathcal{X}$,*

$$D_\alpha\big(p_{\mathcal{M}(x)} \,\|\, p_{\mathcal{M}(x')}\big) \leq \epsilon\, d(x, x'),$$

where $\alpha > 1$, $\epsilon \geq 0$, and $p_{\mathcal{M}(x)}$ denotes the distribution of the mechanism output on input x.

For LLM inference, RmDP offers two key advantages:

- RmDP enables quantifying the privacy protection of clients' sensitive tokens by defining a metric on $\mathbb{Z}_n^l$.
- By leveraging the α-Rényi divergence, RmDP can handle high-dimensional Gaussian noise without resorting to a relaxed variant.

Table 1 compares DP, Rényi DP, metric DP, and Rényi-metric DP.

6.2 Security Analysis of Private Tokens

To characterize the privacy guarantee on token-sequence space $\mathbb{Z}_n^l$ with RmDP, we define a metric in Definition 2.

Definition 2 (Permutation Metric on $\mathbb{Z}_n^l$)**.** *For any two token sequences $x, x' \in \mathbb{Z}_n^l$, define $d(x, x')$ as the minimum number of transpositions needed to transform x into x'. More precisely, $d(x, x')$ is the smallest integer k such that there exists a sequence*

$$x = x_0 \to x_1 \to \cdots \to x_k = x'$$

Table 1: Comparison of DP, Rényi DP, metric DP, and Rényi-metric DP.

	DP	Rényi DP	Metric DP	Rényi-metric DP
Privacy notion	Adjacent datasets	Adjacent datasets	Metric space	Metric space
Gaussian mechanism analysis	Not natural	Natural	Not natural	Natural

where, for each $i = 0, \ldots, k-1$, there exists a transposition (a permutation that swaps exactly two elements) $g_i \in S_n$ satisfying $x_{i+1} = g_i x_i$.

As stated in Section 5.3, `AloePri` is compatible with data obfuscation techniques based on token perturbation. We consider a standard exponential mechanism for token perturbation, denoted by $\mathcal{M}_1$, satisfying

$$p_{\mathcal{M}_1(x)}(y) \propto e^{-\epsilon_1 d(x,y)}.$$

We then derive the privacy guarantee of `AloePri` in Theorem 4 (full proof in Appendix C) when applying $\mathcal{M}_1$ in the online phase.

Theorem 4. *Let $d(\cdot,\cdot)$ be the metric in Definition 2. `AloePri` satisfies (α-ϵ-d)-RmDP, where $\alpha = 2$, and*

$$\epsilon = \begin{cases} \epsilon_1 - \frac{\epsilon_1^2}{4(n-1)\epsilon_2}, & \text{if} \epsilon_1 \leq 2(n-1)\epsilon_2 \\ (n-1)\epsilon_2, & \text{otherwise} \end{cases}$$

Here

$$\epsilon_2 = \pi^2 \cdot (\epsilon_e + \epsilon_h), \epsilon_e = \frac{\alpha\left(\lambda_1^2(W_e) + \lambda_2^2(W_e)\right)}{4\sigma_e^2}, \epsilon_h = \frac{\alpha\left(\lambda_1^2(W_h) + \lambda_2^2(W_h)\right)}{4\sigma_h^2},$$

where $\lambda_1(W_e) \geq \lambda_2(W_e)$ are the largest and second-largest singular values of W_e, and $\lambda_1(W_h) \geq \lambda_2(W_h)$ are the largest and second-largest singular values of W_h.

Remark 1. *`AloePri`'s consumed privacy budget ϵ is strictly smaller than the consumed privacy budget ϵ_1 of data obfuscation $\mathcal{M}_1$.*

Theorem 4 demonstrates that the privacy protection of `AloePri` exhibits two key properties. When the data obfuscation mechanism $\mathcal{M}_1$ provides satisfactory privacy guarantees (i.e., $\epsilon_1 \leq (n-1)\epsilon_2$), `AloePri` can further reduce the privacy budget. When the privacy protection offered by $\mathcal{M}_1$ is limited, `AloePri` still ensures smaller privacy budget consumption through model obfuscation.

7 Experiments

In this section, we present the experimental results of `AloePri` and their explanation.

7.1 Experimental Settings

Models and Datasets. We conduct experiments on commonly-used open-source LLMs to evaluate the effectiveness of `AloePri`. For dense models, we select Qwen2.5/Qwen3 [49, 50], Llama3 [51], Deepseek-R1-Distill-Qwen (R1-Distill) [43]. For MoE models, we choose Qwen3-MoE and Deepseek-V3.1-Terminus [52], covering both moderate and large-scale models.

We employ various representative LLM evaluation benchmarks to evaluate model accuracy, including SST2 [53] for sentence classification, MMLU [54] and C-Eval [55] for general tasks, HumanEval [56] for code generation, IFEval [57] for alignment, and PIQA [58] for physical commonsense reasoning. Additionally, we use PUPA dataset [14] to validate the protection level for Personally Identifiable Information (PII). We use CCI3 [59], Huatuo26M-Lite [60], and MedDialog [61] datasets to evaluate information leakage related to token frequency patterns.

Baseline Methods. According to the discussion in Section 2, we compare `AloePri` with four obfuscation-based privacy-preserving inference methods, which are the most related to our work. `SANTEXT` [23] and `RANTEXT` [11] are based on token substitution, while `DP-Forward` [16] and `SGT` [17] are based on embedding transformation. Because cryptography-based methods are still far from practical usage and TEE-based methods lack hardware compatibility, we do not compare `AloePri` with them.

Attacks. We evaluate the resistance of `AloePri` against three types of typical attacks. (1) Obfuscation recovery: We adopt Vocabulary-Matching Attack (VMA) [25] and Invariant Attack (IA) [34] to verify whether the attacker can

recover the secret mapping of AloePri based on the relationship between plaintext and obfuscated mode weights. (2) Training-based inversion: We use Internal State Attack (ISA) [24], Inversion Model Attack (IMA) [26], and nearest neighbor (NN) attack to investigate the protection of internal states. (3) Token frequency exploit: We use Token Frequency Matching Attack (TFMA) and Substitution Deciphering Attack (SDA) [27] to test the information leakage of token frequency. To the best of our knowledge, these attacks encompass almost all privacy threats faced by obfuscation-based privacy-preserving inference methods. We provide detailed information about the above attacks in Appendix D.1.

Privacy Metrics. We adopt several metrics to quantify the privacy level. The *Text Token Recovery Success Ratio (TTRSR)* measures the proportion of original tokens successfully recovered by the attacker, providing a direct measure of token-level privacy protection. The *PII Recovery Success Ratio (PIIRSR)* quantifies the percentage of personally identifiable information (e.g., names, addresses, phone numbers) correctly extracted from the obfuscated outputs. The *Recovered Text Cosine Similarity (CosSim)* calculates the cosine similarity between the embedding of the recovered text and the original plaintext. *Top-k* accuracy assesses the accuracy of token matching based on the k-nearest neighbors. The *BLEU-4* score measures the n-gram overlap between the adversary's recovered text and the original plaintext.

Hyperparameter Configuration. In accordance with the recommended configurations of these models, we fix the generation parameters across all experiments with temperature = 0.65, top-k = 20, and top-p = 0.95. As for the privacy parameters, we tune them in subsequent experiments. If not specified, we set the matrix coefficient to $\lambda = 0.3$, the expansion size to $h = 128$, the noise coefficients to $\alpha_e = 1.0$ and $\alpha_h = 0.2$, and the attention block-wise parameters to $\beta = 8$ and $\gamma = 1e^3$ by default. We present detailed hyperparameter settings of subsequent experiments in Appendix D.2.

Table 2: Accuracy and privacy comparison with baselines on Qwen2.5-14B-Instruct.

Method	Classification Acc. ↑	Generation Acc. ↑			Privacy[1]		
	SST2	MMLU	PIQA	IFEval	Attack	TTRSR(%) ↓	CosSim ↓
Plaintext	97.25	81.95	77.45	77.09	-	-	-
SANTEXT	81.31	51.07	49.72	57.45	(Direct obs.)	62.90	0.93
RANTEXT	87.50	37.77	18.30	57.18	(Direct obs.)	56.92	0.88
DP-Forward[2]	93.60	-	-	-	NN	37.82	0.51
					IMA	77.08	0.84
SGT[3]	92.43	30.30	26.06	34.66	NN	11.12	0.69
					IMA	98.58	0.99
AloePri	**97.13**	**80.61**	**75.23**	**79.49**	VMA	13.51	0.31
					NN	0.0	0.42
					IMA	0.0	0.36
					IA	5.95	0.40
					ISA	0.0	0.20

[1] Only count the privacy of the input text, as the baselines do not protect generated texts.
[2] DP-Forward cannot support for auto-regressive generation tasks.
[3] SGT does not release source codes. Therefore, we re-implement SGT according to the description in their paper [17].

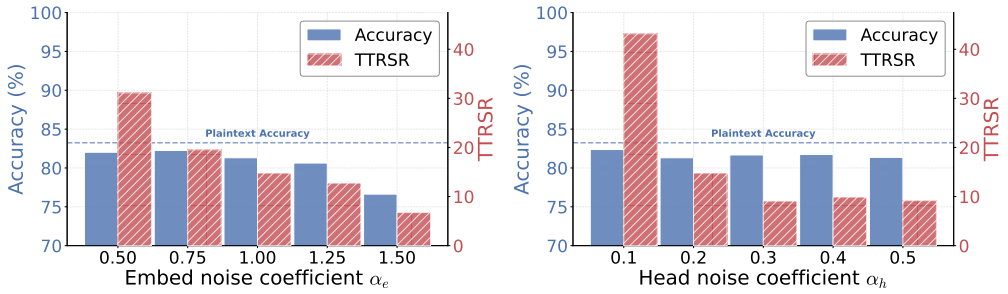
Experimental Environment. The client-side environment is configured with a CPU-only setup equipped with two Intel(R) Xeon(R) Platinum 8457C processors (96 cores). The server-side environment employs a high-performance GPU cluster to handle the computational demands of large language model inference. We deploy the server-side inference service with vLLM 0.9.1 [29] and CUDA 12.4.

7.2 Comparison with Previous Methods

We compared AloePri with data obfuscation methods on model accuracy and privacy. The model accuracy is evaluated on MMLU, IFEval, and PIQA. We conducted various attacks (e.g. NN, IMA, VMA, IA, and ISA) to test privacy levels of different methods. We present the details and hyperparameter settings of baselines in Appendix D.3.

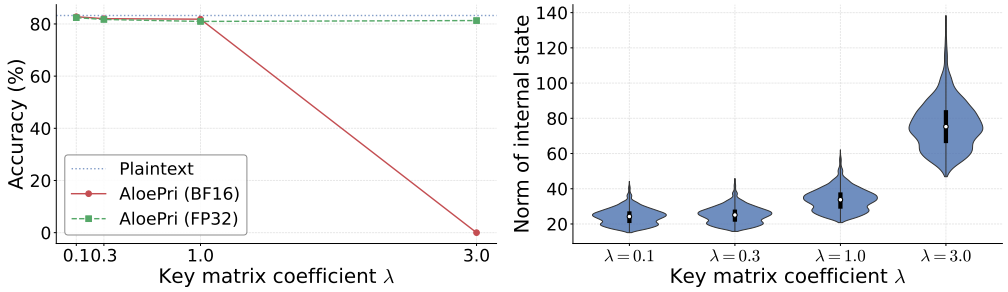
As shown in Table 2, AloePri significantly outperforms baselines in both model accuracy and data privacy. For SANTEXT and RANTEXT, we calculated the TTRSR by counting the number of plaintext tokens directly observed by the attacker. The two methods leak over 50% of tokens while still suffer significant accuracy degradation. The reason is that the sensitive tokens replacement directly disrupts the key semantic information needed for answer generation. DP-Forward reaches about 94% accuracy on SST2, but it is vulnerable to IMA with over 75% TTRSR. In SGT, the

client leverages a 1.6B-parameter model for noise generation. However, SGT still causes over 50% accuracy loss. Meanwhile, SGT is also vulnerable to IMA, leading to over 90% TTRSR. Based on covariant obfuscation, AloePri achieves a better balance between model accuracy and privacy. AloePri shows strong resistance to VMA, IMA, IA, and ISA, i.e., TTRSR is less than 15%. Meanwhile, AloePri achieves less than 3% accuracy loss on both classification and generation tasks.



(a) Various α_e with fixed $\alpha_h = 0.2$ (b) Various α_h with fixed $\alpha_e = 1.0$

Figure 3: Privacy (TTRSR) and accuracy under various noise parameters on Qwen2.5-14B-Instruct and C-Eval.



(a) Accuracy of C-Eval. (b) Norm distribution of internal states.

Figure 4: Impact of λ on Qwen2.5-14B-Instruct.

Table 3: Accuracy and privacy of AloePri under various models and datasets. Privacy is evaluated with VMA on PUPA dataset.

Model		Plaintext accuracy ↑					AloePri accuracy ↑					AloePri privacy against VMA ↓		
		MMLU	C-Eval	HumanEval	PIQA	IFEval	MMLU	C-Eval	HumanEval	PIQA	IFEval	TTRSR(%)	PIIRSR(%)	BLEU-4
R1-Distill	14B	78.91	86.18	96.34	88.14	72.83	79.2	85.93	93.9	88.79	72.27	12.36	2.37	0.52
	32B	83.5	88.34	97.56	88.79	73.94	83.61	87.84	95.12	88.08	73.57	10.10	2.63	0.78
Qwen3	14B	87.64	87.35	94.51	89.72	86.14	84.57	87.12	95.12	90.26	85.40	25.05	1.62	1.72
	32B	89.61	89.0	98.78	90.75	84.29	86.24	87.64	96.95	89.5	81.15	19.64	1.23	1.41
Llama3	8B	65.91	50.16	55.49	80.74	68.58	63.5	44.55	57.32	78.94	67.84	2.57	0	0.84
Qwen3-MoE-Instruct	30B-A3B	84.75	86.97	85.37	89.55	80.41	84.49	86.79	83.54	82.21	78.56	5.91	2.43	0.74
DS-V3.1-Terminus (no_think)	671B	91.72	87.67	94.51	83.84	86.69	90.67	85.65	95.12	80.96	84.66	4.80	1.12	0.40

7.3 Impact of Privacy Parameters

We then evaluated the accuracy (measured by C-Eval) and privacy (against the VMA attack) of AloePri under different noise parameter settings. As depicted in Figure 3, α_e and α_h both play a critical role in preserving data privacy. For instance, the VMA attack can recover over 30% tokens when α_e is set to 0.5. The reason is that when the noise added to the weights of the embedding or model head is small, each pair of plain and obfuscated weights in Table 9 differs almost only by row-wise and column-wise permutations. Consequently, VMA can easily recover the secret permutation Π through the approach of sorting followed by matching. Considering the trade-off between accuracy and privacy, we recommend setting $\alpha_e = 1.0$ and $\alpha_h = 0.2$ for practical applications.

In Figure 4a, we evaluate the impact of λ on model accuracy for both bfloat16 (BF16) and float32 (FP32) precisions. The results demonstrate that model accuracy degrades drastically at $\lambda = 3.0$ under the BF16 precision setting. Further

observations of the norm distribution of internal states reported in Figure 4b reveal that this phenomenon arises because increasing λ to enhance the obfuscation level of the key matrix simultaneously enlarges the distribution range of internal states, which makes numerical overflow more likely to occur during BF16-precision computations.

7.4 Accuracy and Privacy under Various Models and Datasets

To demonstrate the generality of `AloePri`, we tested the accuracy of `AloePri` on more models and datasets. We tested five commonly used datasets in the LLM benchmark (MMLU, C-Eval, HumanEval, PIQA, IFEval). For privacy, we used PUPA dataset, which is widely used to evaluate privacy levels of previous studies [14].

As shown in Table 3, `AloePri` can strongly protect input data while preserving model accuracy well for various models and tasks. Concretely, the accuracy loss of `AloePri` compared with plaintext model inference is less than 3% in most datasets. Meanwhile, `AloePri` can effectively defend against VMA, i.e., PIIRSR of VMA on `AloePri` are less than 3% for all models. These results further validate the generality of `AloePri` on models with various architectures.

Table 4: Ablative studies for internal state protection against ISA

Applied mechanism	TTRSR(%) ↓	
	AttnScore	HiddenState
Noise	87.14	40.0
Noise + KeyMat	87.14	0.82
Noise + KeyMat + Head&BlockPerm	0.0	0.0

Table 5: Attack success rate of TFMA and SDA.

Attack setting		TFMA(%) ↓		SDA ↓
Pub. data	Priv. data	Top-10	Top-100	BLEU-4
CCI3	Huatuo26M	0.14	1.01	0.01
MedDialog	Huatuo26M	0.43	3.83	0.55
Huatuo26M	Huatuo26M	3.19	16.51	2.10

7.5 Ablative Analysis of Internal States Protection

We conducted ablative experiments to test the protective effect of `AloePri` on internal states (attention scores $AttnScore$ and decoder-layer hidden states $HiddenState$) against ISA. The obfuscation techniques tested include embedding noise $Noise$, key matrices $KeyMat$, attention block, and head permutation $Head\&BlockPerm$. As shown in Table 4, by combining embedding noise, key matrices, and attention block and head permutation, `AloePri` provides strong protection for internal states. Concretely, merely adding embedding noise is insufficient to prevent attackers from recovering tokens via internal states. After applying obfuscation of key matrices, the attacker can hardly exploit the hidden states output by decoder layers to perform ISA. However, attention scores remain vulnerable to ISA, as the attention scores generated by attention queries and keys are not obfuscated by the key matrices. Consequently, in `AloePri`, permutations of attention heads and blocks are introduced to further protect attention scores.

7.6 Information Leakage of Token Frequency

We evaluated attackers' ability to reconstruct private texts from token frequency via TFMA and SDA, defining three attack settings based on their prior knowledge of the client's private data: **Zero-knowledge** (no client data information), where attackers leverage the public corpus CCI3 to recover the client's medical domain dataset Huatuo26M-Lite; **Domain-aware** (access to client data domain information), where attackers use the medical domain dataset MedDiag to recover Huatuo26M-Lite; **Distribution-aware** (knowledge of the client data's specific distribution), where attackers utilize Huatuo26M-Lite to recover a user dataset constructed from a different subset of the same dataset.

As shown in Table 5, in all three settings, `AloePri` can effectively prevent the attacker obtain private data through token frequency. Concretely, even when the attacker has prior knowledge of the user data distribution, it remains challenging to recover the client dataset by exploiting token frequency information. For instance, when using TFMA to directly perform matching based on token frequencies, the attacker's Top-100 token recovery success rate is still no higher than 20%. When employing the transformer-based SDA, the BLEU-4 score for the recovered text quality is only around 2, which is insufficient to form meaningful text.

Table 6: Runtime of Offline Model Obfuscation.

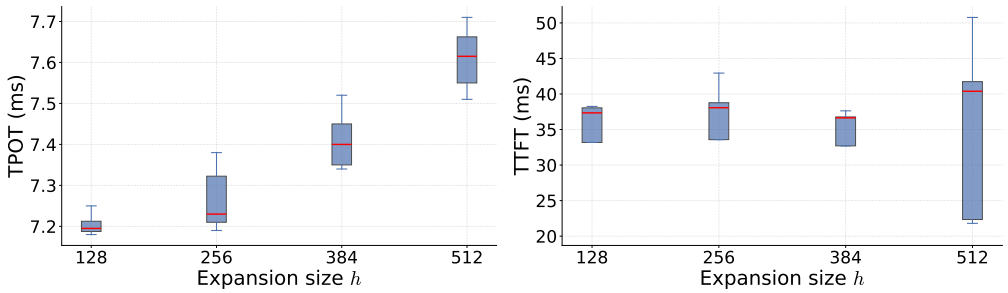
Model	Time cost (min)
R1-Distill-14B	3.43
R1-Distill-32B	9.28
Qwen3-MoE-30B-A3B	4.58
Deepseek-V3.1-Terminus	482.38

Table 7: Online Inference TTFT (ms) and TPOT (ms)

Model	Concurrency	Plaintext		`AloePri`	
		TTFT	TPOT	TTFT	TPOT
R1-Distill-14B	1	19.49	6.93	20.56	6.72
	4	33.34	7.41	36.64	7.22
Deepseek-V3.1-Terminus	1	166.08	19.14	172.04	19.63
	4	185.11	21.42	199.97	22.41

7.7 Efficiency of AloePri

As shown in Table 6, to evaluate the efficiency of AloePri, we test the runtime of offline model obfuscation and online inference on four models. Experimental results show that AloePri delivers practical efficiency for real-world deployment. For offline model obfuscation, it takes only about 10 minutes for a 32B dense model and about 8 hours for the 671B Deepseek-V3.1-Terminus. As this is an offline process, the runtime overhead is acceptable for practical use. For online inference, we compare AloePri with plaintext inference in text generation efficiency using *Time to First Token (TTFT)* and *Time Per Output Token (TPOT)* metrics. We test R1-Distill-14B with tensor parallelism set to 4, average 17-token prompts, 100-token generated texts and varying client request concurrency levels. Results in Table 7 show that AloePri achieves identical online efficiency to plaintext inference.



(a) TPOT (b) TTFT

Figure 5: TPOT and TTFT vs. h on R1-Distill-14B

In Figure 5, we further investigate the impact of the expansion size on inference efficiency using R1-Distill-14B with request concurrency set to 4. Experimental results demonstrate that increasing the expansion size only slightly adds token latency, as modern LLM frameworks incorporate a variety of optimizations for parallel matrix computation. For example, the increase in TPOT is less than 10% even when h is scaled up to 512.

8 Discussion and Future Work

Generalization to Other Models. We focus primarily on text-generative LLMs in this work, but our method's design principles generalize to emerging Transformer variants and other neural network architectures (e.g., multi-modal LLMs, CNNs) under the covariant obfuscation mechanism. Most model architectures can be formulated as directed acyclic graphs (DAGs), with nodes (model components) connected sequentially or in parallel. Thus, the composition theorems of covariant obfuscation remain applicable for designing obfuscation schemes for these models, making carefully designed covariant obfuscation feasible for them.

Protection of Model Weights. AloePri can be further extended to scenarios where only the server can access the model. In this case, CPU-TEE or FHE can be leveraged to allow the client to perform obfuscation without accessing the plaintext model. Taking FHE as an example: the server encrypts model weights via FHE and transmits them to the client, who obfuscates the encrypted weights and returns the results to the server. In this way, model obfuscation is achieved while keeping the model confidential from the client.

9 Conclusion

In this paper, we propose covariant obfuscation and design a privacy-preserving LLM inference method AloePri for industrial applications. We theoretically demonstrate the superiority of covariant obfuscation over data obfuscation methods and propose three composition theorems for covariant obfuscation construction. Subsequently, by constructing covariant obfuscation for each model component and composing them based on composition theorems, AloePri can strongly protect user privacy in LLM inference while preserving model accuracy well. Through comprehensive evaluations, we show that AloePri can maintain high model accuracy while effectively defending against representative attacks. For example, for Deepseek-V3.1-Terminus, with an accuracy loss smaller than 3.5% and similar efficiency to plaintext inference, attackers can only recover less than 5% of text tokens through inversion attacks. Our results demonstrate that utilizing internal structures of the model inference process based on covariant obfuscation could be a better way to design accurate, low-cost, and efficient privacy-preserving LLM inference methods.

References

[1] Claude. Claude 3.5 sonnet, 2024.

[2] Dan Hendrycks and et al. Measuring massive multitask language understanding. In *ICLR*, 2021.

[3] Percy Liang and et al. Holistic evaluation of language models. *arXiv preprint arXiv:2211.09110*, 2022.

[4] Lianmin Zheng and et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *NeurIPS*, 2023.

[5] Robert B Miller. Response time in man-computer conversational transactions. In *Proceedings of fall joint computer conference, part I*, pages 267–277, 1968.

[6] Kanav Gupta, Neha Jawalkar, Ananta Mukherjee, Nishanth Chandran, Divya Gupta, Ashish Panwar, and Rahul Sharma. Sigma: Secure gpt inference with function secret sharing. *Cryptology ePrint Archive*, 2023.

[7] Jiawen Zhang, Xinpeng Yang, Lipeng He, Kejia Chen, Wen-jie Lu, Yinghao Wang, Xiaoyang Hou, Jian Liu, Kui Ren, and Xiaohu Yang. Secure transformer inference made non-interactive. *Cryptology ePrint Archive*, 2024.

[8] Yifan Tan, Cheng Tan, Zeyu Mi, and Haibo Chen. Pipellm: Fast and confidential large language model services with speculative pipelined encryption. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, pages 843–857, 2025.

[9] Lucien K.L. Ng, Sherman S.M. Chow, Anna P.Y. Woo, Donald P. H. Wong, and Yongjun Zhao. Goten: {GPU}-outsourcing trusted execution of neural network training and prediction, 2020.

[10] Ziqi Zhang, Chen Gong, Yifeng Cai, Yuanyuan Yuan, Bingyan Liu, Ding Li, Yao Guo, and Xiangqun Chen. No privacy left outside: On the (in-) security of tee-shielded dnn partition for on-device ml. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 3327–3345. IEEE, 2024.

[11] Meng Tong, Kejiang Chen, Jie Zhang, Yuang Qi, Weiming Zhang, Nenghai Yu, Tianwei Zhang, and Zhikun Zhang. Inferdpt: Privacy-preserving inference for black-box large language models. *IEEE Transactions on Dependable and Secure Computing*, 2025.

[12] Amrita Roy Chowdhury, David Glukhov, Divyam Anshumaan, Prasad Chalasani, Nicolas Papernot, Somesh Jha, and Mihir Bellare. Pr $\epsilon\epsilon$ mpt: Sanitizing sensitive prompts for llms. *arXiv preprint arXiv:2504.05147*, 2025.

[13] Xinyu Tang, Richard Shin, Huseyin A Inan, Andre Manoel, Fatemehsadat Mireshghallah, Zinan Lin, Sivakanth Gopi, Janardhan Kulkarni, and Robert Sim. Privacy-preserving in-context learning with differentially private few-shot generation. *arXiv preprint arXiv:2309.11765*, 2023.

[14] Siyan Li, Vethavikashini Chithrra Raghuram, Omar Khattab, Julia Hirschberg, and Zhou Yu. Papillon: Privacy preservation from internet-based and local language model ensembles. In *NAACL*, 2025.

[15] Zhili Shen, Zihang Xi, Ying He, Wei Tong, Jingyu Hua, and Sheng Zhong. The fire thief is also the keeper: Balancing usability and privacy in prompts. *arXiv preprint arXiv:2406.14318*, 2024.

[16] Minxin Du, Xiang Yue, Sherman SM Chow, Tianhao Wang, Chenyu Huang, and Huan Sun. Dp-forward: Fine-tuning and inference on language models with differential privacy in forward pass. In *CCS*, pages 2665–2679, 2023.

[17] Jay Roberts, Kyle Mylonakis, Sidhartha Roy, and Kaan Kale. Learning obfuscations of llm embedding sequences: Stained glass transform. *arXiv preprint arXiv:2506.09452*, 2025.

[18] Mu Yuan, Lan Zhang, and Xiang-Yang Li. Secure transformer inference protocol. *arXiv preprint arXiv:2312.00025*, 2023.

[19] Khoa Nguyen, Khiem Ton, NhatHai Phan, Issa Khalil, Khang Tran, Cristian Borcea, Ruoming Jin, Abdallah Khreishah, and My T. Thai. Noir: Privacy-preserving generation of code with open-source llms, 2026.

[20] Cynthia Dwork. Differential privacy. In *International colloquium on automata, languages, and programming*, pages 1–12. Springer, 2006.

[21] Ilya Mironov. Rényi differential privacy. In *2017 IEEE 30th computer security foundations symposium (CSF)*, pages 263–275. IEEE, 2017.

[22] Xinpeng Xie, Chenyang Yu, Yan Huang, Yang Cao, and Chenxi Qiu. A decade of metric differential privacy: Advancements and applications, 2025.

[23] Xiang Yue, Minxin Du, Tianhao Wang, Yaliang Li, Huan Sun, and Sherman SM Chow. Differential privacy for text analytics via natural text sanitization. *arXiv preprint arXiv:2106.01221*, 2021.

[24] Tian Dong, Yan Meng, Shaofeng Li, Guoxing Chen, Zhen Liu, and Haojin Zhu. Depth gives a false sense of privacy:{LLM} internal states inversion. In *USENIX Security*, pages 1629–1648, 2025.

[25] Rahul Krishna Thomas, Louai Zahran, Erica Choi, Akilesh Potti, Micah Goldblum, and Arka Pal. Hidden no more: Attacking and defending private third-party LLM inference. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu, editors, *ICML*, volume 267 of *Proceedings of Machine Learning Research*, pages 59434–59469. PMLR, 13–19 Jul 2025.

[26] Kai Kugler, Simon Münker, Johannes Höhmann, and Achim Rettinger. Invbert: Reconstructing text from contextualized word embeddings by inverting the bert pipeline. *arXiv preprint arXiv:2109.10104*, 2021.

[27] Nada Aldarrab and Jonathan May. Can sequence-to-sequence models crack substitution ciphers? In *EMNLP*, pages 7226–7235, 2021.

[28] Nishant Kambhatla, Logan Born, and Anoop Sarkar. Decipherment as regression: Solving historical substitution ciphers by learning symbol recurrence relations. In *EACL*, pages 2136–2152, 2023.

[29] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *OSDI*, 2023.

[30] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. Sglang: Efficient execution of structured language model programs. *NeurIPS*, 37:62557–62583, 2024.

[31] Ziqian Zeng, Jianwei Wang, Junyao Yang, Zhengdong Lu, Haoran Li, Huiping Zhuang, and Cen Chen. Privacyrestore: Privacy-preserving inference in large language models via privacy removal and restoration. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10821–10855, 2025.

[32] Peihua Mai, Ran Yan, Zhe Huang, Youjia Yang, and Yan Pang. Split-and-denoise: Protect large language model inference with local differential privacy. *arXiv preprint arXiv:2310.09130*, 2023.

[33] Sam Lin, Wenyue Hua, Zhenting Wang, Mingyu Jin, Lizhou Fan, and Yongfeng Zhang. Emojiprompt: Generative prompt obfuscation for privacy-preserving communication with cloud-based llms. In *NAACL*, pages 12342–12361, 2025.

[34] Yu Lin, Qizhi Zhang, Quanwei Cai, Jue Hong, Wu Ye, Huiqi Liu, and Bing Duan. An inversion attack against obfuscated embedding matrix in language model inference. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 2100–2104, 2024.

[35] Wen-jie Lu, Zhicong Huang, Zhen Gu, Jingyu Li, Jian Liu, Cheng Hong, Kui Ren, Tao Wei, and WenGuang Chen. Bumblebee: Secure two-party inference framework for large transformers. *Cryptology ePrint Archive*, 2023.

[36] Dacheng Li, Rulin Shao, Hongyi Wang, Han Guo, Eric P Xing, and Hao Zhang. Mpcformer: fast, performant and private transformer inference with mpc. *arXiv preprint arXiv:2211.01452*, 2022.

[37] Chenxu Wang, Danqing Tang, Changxu Ci, Junjie Huang, Yankai Xu, Fengwei Zhang, Jiannong Cao, Jie Song, Shoumeng Yan, Tao Wei, and Zhengyu He. ccai: A compatible and confidential system for ai computing. In *MICRO*, page 340–353, New York, NY, USA, 2025. Association for Computing Machinery.

[38] Florian Tramer and Dan Boneh. Slalom: Fast, verifiable and private execution of neural networks in trusted hardware. In *ICLR*, 2019.

[39] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. Gshard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668*, 2020.

[40] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[41] Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.

[42] Joshua Ainslie, James Lee-Thorp, Michiel De Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023.

[43] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[44] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

[45] Wenliang Du and Zhijun Zhan. A practical approach to solve secure multi-party computation problems. In *Proceedings of the 2002 workshop on New security paradigms*, pages 127–135, 2002.

[46] Milad Nasr, Shuang Songi, Abhradeep Thakurta, Nicolas Papernot, and Nicholas Carlin. Adversary instantiation: Lower bounds for differentially private machine learning. In *2021 IEEE Symposium on security and privacy (SP)*, pages 866–882. IEEE, 2021.

[47] Konstantinos Chatzikokolakis, Miguel E. Andrés, Nicolás Emilio Bordenabe, and Catuscia Palamidessi. Broadening the scope of differential privacy using metrics. In Emiliano De Cristofaro and Matthew Wright, editors, *Privacy Enhancing Technologies*, pages 82–102, Berlin, Heidelberg, 2013. Springer Berlin Heidelberg.

[48] Sai Chen, Fengran Mo, Yanhao Wang, Cen Chen, Jian-Yun Nie, Chengyu Wang, and Jamie Cui. A customized text sanitization mechanism with differential privacy. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 5747–5758, 2023.

[49] Qwen Team. Qwen2.5: A party of foundation models, September 2024.

[50] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[51] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[52] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

[53] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D Manning, Andrew Y Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *NeurIPS*, pages 1631–1642, 2013.

[54] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *ICLR*, 2021.

[55] Yuzhen Huang, Yuzhuo Bai, Zhihao Zhu, Junlei Zhang, Jinghan Zhang, Tangjun Su, Junteng Liu, Chuancheng Lv, Yikai Zhang, Jiayi Lei, Yao Fu, Maosong Sun, and Junxian He. C-eval: A multi-level multi-discipline chinese evaluation suite for foundation models. *arXiv preprint arXiv:2305.08322*, 2023.

[56] Mark Chen. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[57] Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023.

[58] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. Piqa: Reasoning about physical commonsense in natural language. In *AAAI*, 2020.

[59] Liangdong Wang, Bo-Wen Zhang, Chengwei Wu, Hanyu Zhao, Xiaofeng Shi, Shuhao Gu, Jijie Li, Quanyue Ma, TengFei Pan, and Guang Liu. Cci3. 0-hq: a large-scale chinese dataset of high quality designed for pre-training large language models. *arXiv preprint arXiv:2410.18505*, 2024.

[60] Xidong Wang, Jianquan Li, Shunian Chen, Yuxuan Zhu, Xiangbo Wu, Zhiyi Zhang, Xiaolong Xu, Junying Chen, Jie Fu, Xiang Wan, et al. Huatuo-26m, a large-scale chinese medical qa dataset. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 3828–3848, 2025.

[61] Shu Chen, Zeqian Ju, Xiangyu Dong, Hongchao Fang, Sicheng Wang, Yue Yang, Jiaqi Zeng, Ruisi Zhang, Ruoyu Zhang, Meng Zhou, Penghui Zhu, and Pengtao Xie. Meddialog: a large-scale medical dialogue dataset. *arXiv preprint arXiv:2004.03329*, 2020.

[62] Pavel Etingof. Lie groups and lie algebras, 2024.

[63] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *ICLR*, 1(2):3, 2022.

A Notation Table

The notations used in the paper is concluded in Table 8.

Table 8: Notations and their Descriptions

Notation	Description
n	Number of vocabulary's tokens
$d, d_{\text{head}}, d_{\text{ffn}}$	Hidden size, attention head dimension, and FFN intermediate dimension
h	Expansion size for the key matrix
d'	Expanded hidden size
m, m_{kv}	Number of attention heads, and number of attention key-value heads
m_{exp}	Number of MoE experts
$\theta, \mathcal{V}$	Weights and vocabulary of the language model
L	Number of decoder layers
W_e, W_h	Weights of the embedding layer and model head layer
$\boldsymbol{w}_{\text{norm}}$	Weights of the layer normalization
ω_{attn}	Weights of an attention layer (including $W_{\text{query}}, W_{\text{key}}, W_{\text{value}}, W_{\text{out}}$)
ω_{ffn}	Weights of the FFN layer (including $W_{\text{gate}}, W_{\text{up}}, W_{\text{down}}$)
λ	Coefficient for matrix sampling.
$\alpha_{\text{e}}, \alpha_{\text{h}}$	Noise parameters for the embedding layer and model head layer
β, γ	Maximum window size and sampling parameter for the attention block
τ, Π	Secret permutation of the vocabulary and its corresponding permutation matrix
$\hat{P}, \hat{Q}$	Randomly generated key matrix and inverse key matrix

B Proof of Covariant Obfuscation

We present the proofs of composition theorems of covariant obfuscation.

Proof of Theorem 1:

Proof. We separately prove the commutation and de-obfuscation conditions of covariant obfuscation.

A. Commutation Condition: If the error of C_1 is e_{C_1}, the error of C_2 is e_{C_2}, and $\tilde{g}(\tilde{x}, \tilde{\xi})$ satisfies the Lipschitz condition $d(\tilde{g}(\tilde{x}_1, \tilde{\xi}), \tilde{g}(\tilde{x}_2, \tilde{\xi})) \leq M_g d(\tilde{x}_1, \tilde{x}_2)$ with respect to $\tilde{x}$, then we have

$$
\begin{aligned}
&d(\tilde{h}(\phi_X(x), (\phi_\Theta(\theta), \phi_\Xi(\xi))), \phi_Z \circ h(x, (\theta, \xi))) \\
&\leq d(\tilde{g}(\tilde{f}(\phi_X(x), \phi_\Theta(\theta)), \xi), \phi_Z \circ g(f(x, \theta), \xi)) \\
&\leq d(\tilde{g}(\tilde{f}(\phi_X(x), \phi_\Theta(\theta)), \xi), \tilde{g}(\phi_Y \circ f(x, \theta), \xi)) \\
&+ d(\tilde{g}(\phi_Y \circ f(x, \theta), \xi), \phi_Z \circ g(f(x, \theta), \xi))) \\
&\leq M_g d(\tilde{f}(\phi_X(x), \phi_\Theta(\theta)), \phi_Y \circ f(x, \theta)) \\
&+ d(\tilde{g}(\phi_Y \circ f(x, \theta), \xi), \phi_Z \circ g(f(x, \theta), \xi))).
\end{aligned}
$$

Thus $e_{C_2 \circ C_1} \leq M_g e_{C_1} + e_{C_2}$.

B. De-obfuscation Condition: $\psi_Z \circ \phi_Z = \text{id}_Z$ can be directly obtained from the de-obfuscation condition of $(\phi_Y^2, \phi_\Xi, \phi_Z, \psi_Z, \tilde{g})$. □

Proof of Theorem 2:

Proof. A. Commutation Condition:

If the distance function $d_{Y \times Z}$ on $Y \times Z$ satisfies the control condition with respect to the distance functions on Y and Z, then

$$
\begin{aligned}
&d_{Y \times Z}(\tilde{h}(\phi_X(x), (\phi_\Theta(\theta), \phi_\Xi(\xi))), (\phi_Y, \phi_Z) \circ h(x, (\theta, \xi))) \\
\leq &d_Y(\tilde{f}(\phi_X(x), \phi_\Theta(\theta), \phi_Y \circ f(x, \theta)) \\
&+ d_Z(\tilde{g}(\phi_X(x), \phi_\Xi(\xi)), \phi_Z \circ g(x, \xi))).
\end{aligned}
$$

B. De-obfuscation Condition:

$$
(\psi_Y, \psi_Z) \circ (\phi_Y, \phi_Z) = (\psi_Y \circ \phi_Y, \psi_Z \circ \phi_Z) = (\text{id}_Y, \text{id}_Z) = \text{id}_{Y \times Z}.
$$

Proof of Theorem 3:

Proof. A. Commutation Condition:

If the distance function on Y satisfies translation invariance, then for any $a_1, a_2, b_1, b_2 \in Z$, we have

$$\begin{aligned} d(a_1 + b_1, a_2 + b_2) \leq & d(a_1 + b_1, a_2 + b_1) + d(a_2 + b_1, a_2 + b_2) \\ = & d(a_1, a_2) + d(b_1, b_2). \end{aligned}$$

Thus,

$$\begin{aligned} & d_Y(\tilde{h}(\phi_X(x), (\phi_\Theta(\theta), \phi_\Xi(\xi))), \phi_Y \circ h(x, (\theta, \xi))) \\ = & d_Y(\tilde{f}(\phi_X(x), \phi_\Theta(\theta)) + \tilde{g}(\phi_X(x), \phi_\Xi(\xi)), \\ & \phi_Y(f(x, \theta) + g(x, \xi))) \\ \leq & d_Y(\tilde{f}(\phi_X(x), \phi_\Theta(\theta)), \phi_Y \circ f(x, \theta)) \\ & + d_Y(\tilde{g}(\phi_X(x), \phi_\Xi(\xi)), \phi_Y \circ g(x, \xi)). \end{aligned}$$

Therefore, $e_{C_1+C_2} \leq e_{C_1} + e_{C_2}$.

B. De-obfuscation Condition: Directly follows from the de-obfuscation conditions of $(\phi_X, \phi_\Theta, \phi_Z, \psi_Z, \tilde{f})$ and $(\phi_Y, \phi_\Xi, \phi_Z, \psi_Z, \tilde{g})$.

C Proof of Theorem 4

C.1 Rényi Squared Metric Differential Privacy

We first extend RmDP to Rényi squared metric Differential Privacy (RsmDP).

Definition 3 (Rényi squared metric Differential Privacy)**.** *Let $(\mathcal{X}, d)$ be a metric space. A randomized mechanism $\mathcal{M} \colon \mathcal{X} \to \mathcal{Y}$ satisfies* (α, ϵ, d^2)-RsmDP *if, for any $x, x' \in \mathcal{X}$,*

$$D_\alpha\big(p_{\mathcal{M}(x)} \,\|\, p_{\mathcal{M}(x')}\big) \leq \epsilon\, d(x, x')^2,$$

where $\alpha > 1$, $\epsilon \geq 0$, and $p_{\mathcal{M}(x)}$ denotes the distribution of the mechanism output on input x.

C.2 Gauss Mechanism

Theorem 5 (Gaussian Mechanism for RsmDP)**.** *Let $(\mathcal{X}, d)$ be a metric space, and let $f : \mathcal{X} \to \mathbb{R}^d$ be a query function satisfying*

$$L_2(f) := \max_{x, x' \in \mathcal{X}} \frac{\|\Sigma^{-1/2}(f(x) - f(x'))\|_2}{d(x, x')} < \infty,$$

where Σ is a symmetric positive definite matrix.

The Gaussian mechanism is defined by adding Gaussian noise $\mathcal{N}(0, \sigma^2\Sigma)$ to the query output:

$$\mathcal{M}(x) = f(x) + \mathcal{N}(0, \sigma^2\Sigma).$$

This mechanism satisfies (α, ϵ, d^2)-RsmDP with

$$\epsilon = \frac{\alpha L_2(f)^2}{2\sigma^2}.$$

Proof. Let $z := f(x) + \mathcal{N}(0, \sigma^2\Sigma)$. For any $x, x' \in \mathcal{X}$, we have

$$p_{z|x}(z) \sim \mathcal{N}(f(x), \sigma^2\Sigma), \quad p_{z|x'}(z) \sim \mathcal{N}(f(x'), \sigma^2\Sigma).$$

Hence we obtain

$$\begin{aligned} D_\alpha(p_{z|x} \parallel p_{z|x'}) &= \frac{\alpha}{2}(f(x) - f(x'))^\top \sigma^{-2}\Sigma^{-1}(f(x) - f(x')) \\ &= \frac{\alpha}{2\sigma^2} \cdot \frac{\|\Sigma^{-1/2}(f(x) - f(x'))\|_2^2}{d(x, x')^2} \cdot d(x, x')^2 \\ &\leq \frac{\alpha L_2(f)^2}{2\sigma^2}\, d(x, x')^2 = \epsilon\, d(x, x')^2. \end{aligned}$$

□

Theorem 6 (Gaussian Mechanism on Matrix Space for RsmDP)**.** *Let $\Theta := M_{n,d}$. Define the randomized mechanism $M : O_n \to \Theta$ by*

$$M(g) := g(\theta + E),$$

where each row of E is independently and identically distributed as $\mathcal{N}(0, \sigma^2 I)$. Then the following statements hold:

1. *M satisfies (α, ϵ, d^2)-RsmDP for any $\alpha > 1$, where*

$$L_2(f) := \max_{g,g' \in O_n,\ g \neq g'} \frac{\|g\theta - g'\theta\|_2}{d(g,g')}, \qquad \epsilon = \frac{\alpha L_2(f)^2}{2\sigma^2}.$$

2. *If $d(g,g') := \|g - g'\|$, then $L_2(f) = \lambda_1$, where λ_1 is the largest singular value of θ.*
3. *If $d(g,g')$ is the geodesic distance [62], then*

$$L_2(f) \leq \sqrt{\frac{\lambda_1^2 + \lambda_2^2}{2}},$$

where $\lambda_1 \geq \lambda_2$ are the largest and second-largest singular values of θ.

Proof. We prove each statement separately:

1. This follows directly from Theorem 5 by setting $f(g) := g\theta$.
2. Since
$$\|g\theta - g'\theta\| \leq \|\theta\|_{OP} \|g - g'\|,$$
where $\|\theta\|_{OP}$ denotes the operator norm of θ, which is equal to its largest singular value λ_1, we have
$$L_2(f) = \|\theta\|_{OP} = \lambda_1.$$
3. Let $X := \log(g^{-1}g')$, which is a skew-symmetric matrix. Let $A := \theta$. Then
$$\|g\theta - g'\theta\|^2 = \|g^{-1}g'A - A\|^2 = \|e^X A - A\|^2.$$
We have
$$\|e^X A - A\|^2 \leq \|XA\|^2 \leq \frac{\lambda_1^2 + \lambda_2^2}{2}\|X\|^2.$$
Therefore,
$$\|g\theta - g'\theta\|^2 \leq \frac{\lambda_1^2 + \lambda_2^2}{2}\|X\|^2 = \frac{\lambda_1^2 + \lambda_2^2}{2} d(g,g')^2,$$
which implies
$$\frac{\|g\theta - g'\theta\|}{d(g,g')} \leq \sqrt{\frac{\lambda_1^2 + \lambda_2^2}{2}}.$$
Since this holds for all g, g', we conclude that
$$L_2(f) \leq \sqrt{\frac{\lambda_1^2 + \lambda_2^2}{2}}.$$

□

C.3 Exponential Mechanism

Theorem 7 (Exponential Mechanism for RmDP)**.** *For a metric space (X, d) where X is a finite set, define the mechanism $\mathcal{M} : X \to X$ by $\mathcal{M}(x) = y$, where y is a random variable on X with probability distribution satisfying $p_{\mathcal{M}}(y|x) \propto e^{-\epsilon d(x,y)}$. The mechanism $\mathcal{M}$ satisfies (α, ϵ, d)-RmDP for any $\alpha > 1$.*

Proof. We have

$$\left(\frac{p_{\mathcal{M}}(y|x)}{p_{\mathcal{M}}(y|x')}\right)^{\alpha-1} = \exp\left[\epsilon(\alpha-1)\left(d(x',y) - d(x,y)\right)\right] \leq \exp\left[\epsilon(\alpha-1)d(x,x')\right].$$

Therefore,

$$D_\alpha\big(p_{\mathcal{M}}(y|x) \parallel p_{\mathcal{M}}(y|x')\big) = \frac{1}{\alpha}\log \int \left(\frac{p_{\mathcal{M}}(y|x)}{p_{\mathcal{M}}(y|x')}\right)^{\alpha-1} p_{\mathcal{M}}(y|x)dy \leq \frac{1}{\alpha-1}\log\exp\left[\epsilon(\alpha-1)d(x,x')\right] = \epsilon d(x,x').$$

□

C.4 Permutation Metric on S_n

Define the permutation metric $d_p(g, g')$ on S_n as the minimum number k of transpositions required to transform g into g', i.e.,

$$g = g_0 \to g_1 \to \cdots \to g_k = g'.$$

It is easy to see that the permutation metric on $\mathbb{Z}_n^l$ presented in Definition 2 is related to that on S_n as follows:

$$d(x, x') = \min_{g \in S_n:\, gx = x'} d_p(g, \mathrm{id}),$$

where id denotes the identity permutation.

Moreover, we establish the relationship between the permutation metric and the geodesic metric in Theorem 8.

Theorem 8 (Geodesic Metric vs. Permutation Metric)**.** *We have*

1. *For any $g, g' \in S_n$,*
$$d_g(g, g') \le \pi\, d_p(g, g'),$$
where d_g denotes the geodesic metric on the unitary group U_n, and d_p denotes the permutation metric on S_n.
2. *If a mechanism on S_n satisfies $(\alpha, \epsilon, d_g^2)$-RsmDP, it also satisfies $(\alpha, \epsilon\pi^2, d_p^2)$-RsmDP.*

Proof. We prove each statement separately:

1. Suppose $d_p(g, g') = k$. Then there exists a chain of transpositions
$$g = g_0 \to g_1 \to \cdots \to g_k = g'.$$
By the triangle inequality,
$$d_g(g, g') \le \sum_{i=0}^{k-1} d_g(g_i, g_{i+1}) = \sum_{i=0}^{k-1} \pi = k\pi = \pi d_p(g, g').$$

2. By assumption,
$$D_\alpha\big(p(y|x) \,\|\, p(y|x')\big) \le \epsilon\, d_g(x, x')^2.$$
Combining this with the result in part (1), we obtain
$$d_g(x, x')^2 \le \pi^2 d_p(x, x')^2,$$
which implies
$$D_\alpha\big(p(y|x) \,\|\, p(y|x')\big) \le \epsilon\pi^2 d_p(x, x')^2.$$

□

C.5 RsmDP for `AloePri`'s Offline Phase

Theorem 9. *The offline phase of `AloePri` satisfies (α, ϵ, d^2)-RsmDP for protecting the secret permutation τ, where $\alpha = 2$, $\epsilon = \epsilon_e + \epsilon_h$, and d is the geodesic metric on $S_n \subset U_n$. Specifically,*

$$\epsilon_e = \frac{\alpha\big(\lambda_1^2(W_e) + \lambda_2^2(W_e)\big)}{4\sigma_e^2},$$

where $\lambda_1(W_e) \ge \lambda_2(W_e)$ are the largest and second-largest singular values of W_e, and

$$\epsilon_h = \frac{\alpha\big(\lambda_1^2(W_h) + \lambda_2^2(W_h)\big)}{4\sigma_h^2},$$

where $\lambda_1(W_h) \ge \lambda_2(W_h)$ are the largest and second-largest singular values of W_h.

Proof. First, decompose the model parameter space into the embedding part, the middle part, and the model head part:

$$\Theta = \Theta_e \oplus \Theta_{\text{middle}} \oplus \Theta_h.$$

Correspondingly, let $\theta = (W_e, \theta_{\text{middle}}, W_h)$. Then the offline mechanism can be expressed as:

$$\mathcal{M} : S_n \to \Theta, \quad g \mapsto \left(g(\theta_e + E)P, \tilde{\mathcal{M}}(P), g(\theta_h + E)P^{-1}\right)$$

for some random P and sub-mechanism $\tilde{\mathcal{M}}$.

This mechanism can be decomposed into two parts: $\mathcal{M}(g) = \mathcal{M}_2 \circ \mathcal{M}_1(g)$, where

$$\mathcal{M}_1(g) = (g(\theta_e + E), g(\theta_h + E)), \quad \mathcal{M}_2(A, B) = \left(AP, \tilde{\mathcal{M}}(P), BP^{-1}\right).$$

By the data processing inequality,

$$D_\alpha\big(p_{\mathcal{M}}(z|g) \parallel p_{\mathcal{M}}(z|g')\big) \le D_\alpha\big(p_{\mathcal{M}_1}(y|g) \parallel p_{\mathcal{M}_1}(y|g')\big).$$

On the other hand, by Theorem 6, we know that

$$D_\alpha\big(p_{\mathcal{M}_1}(y|g) \parallel p_{\mathcal{M}_1}(y|g')\big) \le \epsilon d^2(g, g'),$$

where $\epsilon = \epsilon_e + \epsilon_h$, d is the geodesic metric on $S_n \subset U_n$,

$$\epsilon_e = \frac{\alpha\left(\lambda_1^2(W_e) + \lambda_2^2(W_e)\right)}{4\sigma_e^2},$$

where $\lambda_1(W_e) \ge \lambda_2(W_e)$ are the largest and second-largest singular values of W_e;

$$\epsilon_h = \frac{\alpha\left(\lambda_1^2(W_h) + \lambda_2^2(W_h)\right)}{4\sigma_h^2},$$

where $\lambda_1(W_h) \ge \lambda_2(W_h)$ are the largest and second-largest singular values of W_h.

Therefore,

$$D_\alpha\big(p_{\mathcal{M}}(z|g) \parallel p_{\mathcal{M}}(z|g')\big) \le \epsilon d^2(g, g').$$

□

Corollary 9.1. *The offline component satisfies (α, ϵ, d^2)-RDP, where $\alpha = 2$, $\epsilon = \pi^2 \cdot (\epsilon_e + \epsilon_h)$, and d is the permutation metric on S_n.*

$$\epsilon_e = \frac{\alpha\left(\lambda_1^2(W_e) + \lambda_2^2(W_e)\right)}{4\sigma_e^2},$$

where $\lambda_1(W_e) \ge \lambda_2(W_e)$ are the largest and second-largest singular values of W_e;

$$\epsilon_h = \frac{\alpha\left(\lambda_1^2(W_h) + \lambda_2^2(W_h)\right)}{4\sigma_h^2},$$

where $\lambda_1(W_h) \ge \lambda_2(W_h)$ are the largest and second-largest singular values of W_h.

Proof. Follows directly from the Theorem 8.

□

C.6 Composability Theorem for Sub-Mechanisms on Token Sequence Space

Let $\mathbb{Z}_n^l$ denote the space of token sequences of length l, and let $d(\cdot, \cdot)$ denote the permutation metric on $\mathbb{Z}_n^l$.

For the exponential mechanism $\mathcal{M}_1 : \mathbb{Z}_n^l \to \mathbb{Z}_n^l$ and the mechanism $\mathcal{M}_2 : S_n \to \Theta$, define the covariance obfuscated exponential mechanism $\mathcal{M} : \mathbb{Z}_n^l \to \mathbb{Z}_n^l \oplus \Theta$ as:

$$\mathcal{M}(x) := (g\mathcal{M}_1(x), \mathcal{M}_2(g)),$$

where $g \in S_n$ is uniformly distributed.

Note: It is easy to see that $\mathcal{M}(x)$ can also be written as $\mathcal{M}(x) := (\mathcal{M}_1(gx), \mathcal{M}_2(g))$.

Theorem 10 (Composability for Sub-Mechanisms on Token Sequence Space)**.** *If the exponential mechanism $\mathcal{M}_1 : \mathbb{Z}_n^l \to \mathbb{Z}_n^l$ satisfies (α, ϵ_1, d)-privacy, and the mechanism $\mathcal{M}_2 : S_n \to \Theta$ satisfies α-ϵ_2-d^2-privacy, where $\alpha = 2$, $\epsilon_1, \epsilon_2 \in \mathbb{R}^+$, and $d(\cdot, \cdot)$ is the permutation metric, then the mechanism $\mathcal{M}$ satisfies α-ϵ-d-privacy, where*

$$\epsilon = \begin{cases} \epsilon_1 - \dfrac{\epsilon_1^2}{4(n-1)\epsilon_2}, & \text{if } \epsilon_1 \le 2(n-1)\epsilon_2, \\ (n-1)\epsilon_2, & \text{otherwise}. \end{cases}$$

From this theorem we can see that:

1. If the sub-mechanisms consume less privacy budget (i.e., permutation recovery is inaccurate), then the privacy budget consumed by the parent mechanism will also decrease accordingly.
2. Even if the sub-mechanisms consume a large privacy budget (i.e., permutation recovery is accurate), the privacy budget consumed by the parent mechanism is still strictly smaller than that of token obfuscation.

Proof. By the convexity of χ^2-divergence, we have

$$D_{\chi^2}\big(p_{\mathcal{M}}(z|x) \parallel p_{\mathcal{M}}(z|x')\big) = D_{\chi^2}\big(E_{g,g'} p_{\mathcal{M}}(z|x,g) \parallel E_{g,g'} p_{\mathcal{M}}(z|x',g')\big) \le E_{g,g'} D_{\chi^2}\big(p_{\mathcal{M}}(z|x,g) \parallel p_{\mathcal{M}}(z|x',g')\big)$$

for any joint distribution of random variables (g, g') with uniform marginal distributions.

Since

$$p_{\mathcal{M}}(z|x,g) = p_{\mathcal{M}_1}(z_1|g,x) \cdot p_{\mathcal{M}_2}(z_2|g),$$

it follows that

$$D_2\big(p_{\mathcal{M}}(z|x,g) \parallel p_{\mathcal{M}}(z|x',g')\big) = D_2\big(p_{\mathcal{M}_1}(z_1|g,x) \parallel p_{\mathcal{M}_1}(z_1|g',x')\big) + D_2\big(p_{\mathcal{M}_2}(z_2|g) \parallel p_{\mathcal{M}_2}(z_2|g')\big).$$

By assumption, $D_2\big(p_{\mathcal{M}_2}(z_2|g) \parallel p_{\mathcal{M}_2}(z_2|g')\big) \le \epsilon_2 d^2(g, g')$, so

$$D_2\big(p_{\mathcal{M}}(z|x,g) \parallel p_{\mathcal{M}}(z|x',g')\big) \le D_2\big(p_{\mathcal{M}_1}(z_1|g,x) \parallel p_{\mathcal{M}_1}(z_1|g',x')\big) + \epsilon_2 d^2(g, g').$$

Furthermore, by the Theorem 7 , we have

$$D_2\big(p_{\mathcal{M}_1}(z_1|g,x) \parallel p_{\mathcal{M}_1}(z_1|g',x')\big) \le \epsilon_1 d(x, g'gx') = \epsilon_1 d(x, \Delta g x'),$$

where $\Delta g = g^{-1}g'$. Thus,

$$D_2\big(p_{\mathcal{M}}(z|x,g) \parallel p_{\mathcal{M}}(z|x',g')\big) \le \epsilon_1 d(x, \Delta g x') + \epsilon_2 d^2(g, g') \quad \text{for any } \Delta g \in S_n.$$

Assume $d(x, x') = r$. Then there exists $h \in S_n$ such that $x' = hx$ and $d(h, 1) = r$. Thus, there exists a chain of transpositions

$$1 = h_0 \to h_1 \to \cdots \to h_r = h.$$

For any $i = 0, 1, \ldots, r$, there exists $\Delta g_i := h_{r-i} h^{-1}$ such that

$$d(x, \Delta g_i x') = d(x, h_{r-i} h^{-1} h x) = d(x, h_{r-i} x) = r - i,$$
$$d(g, g') = d(\Delta g_i, 1) = d(h_{r-i} h^{-1}, 1) = i.$$

Therefore,

$$D_2\big(p_{\mathcal{M}}(z|x,g) \parallel p_{\mathcal{M}}(z|x',g')\big) \le \epsilon_1 (r - i) + \epsilon_2 i^2 =: f_r(i).$$

$f_r(i)$ is a quadratic function in i. Analysis shows that

$$\min_{i=0}^{r} f_r(i) \le \phi(r),$$

where

$$\phi(r) = \begin{cases} \epsilon_2 r^2, & \text{if } r \le \left\lfloor \frac{\epsilon_1}{2\epsilon_2} \right\rfloor, \\ f\left(\left\lfloor \frac{\epsilon_1}{2\epsilon_2} \right\rfloor\right) \approx \epsilon_1 r - \frac{\epsilon_1^2}{4\epsilon_2}, & \text{otherwise.} \end{cases}$$

Considering the range of r is $[0, n-1]$, we have $\phi(r) \le \epsilon r$, where

$$\epsilon = \begin{cases} \epsilon_1 - \dfrac{\epsilon_1^2}{4(n-1)\epsilon_2}, & \text{if } \epsilon_1 \le 2(n-1)\epsilon_2, \\ (n-1)\epsilon_2, & \text{otherwise.} \end{cases}$$

Hence,

$$D_2\big(p_{\mathcal{M}}(z|x,g) \parallel p_{\mathcal{M}}(z|x',g')\big) \le \epsilon d(x,x').$$

It follows that

$$D_{\chi^2}\big(p_{\mathcal{M}}(z|x,g) \parallel p_{\mathcal{M}}(z|x',g')\big) \le \exp\big(\epsilon d(x,x')\big) - 1,$$

and thus

$$D_{\chi^2}\big(p_{\mathcal{M}}(z|x) \parallel p_{\mathcal{M}}(z|x')\big) \le E_{g,g'} D_{\chi^2}\big(p_{\mathcal{M}}(z|x,g) \parallel p_{\mathcal{M}}(z|x',g')\big) \le \exp\big(\epsilon d(x,x')\big) - 1.$$

Therefore,

$$D_2\big(p_{\mathcal{M}}(z|x) \parallel p_{\mathcal{M}}(z|x')\big) \le \epsilon d(x,x').$$

□

C.7 Proof of Theorem 4

Proof. The offline phase satisfies $(\alpha, \epsilon_2, d^2)$-RsmDP with $\alpha = 2$, where

$$\epsilon_2 = \pi^2 \cdot (\epsilon_e + \epsilon_h),$$

and d denotes the permutation metric on S_n. The online phase is given by the covariance-obfuscated exponential mechanism. Therefore, by Theorem 10, the overall mechanism $\mathcal{M}$ satisfies (α, ϵ, d)-RmDP, where

$$\epsilon = \begin{cases} \epsilon_1 - \dfrac{\epsilon_1^2}{4(n-1)\epsilon_2}, & \text{if } \epsilon_1 \le 2(n-1)\epsilon_2, \\ (n-1)\epsilon_2, & \text{otherwise.} \end{cases}$$

□

D Experiment Details

D.1 Potential Threats

Vocabulary-Matching Attack (VMA). `AloePri` protect privacy by mapping plaintext tokens of private text to obfuscated tokens via a permutation Π. With knowledge of the obfuscated model weights, attackers can attempt to map each obfuscated token back to its original plaintext token. This threat leverages the Vocabulary-Matching Attack (VMA) proposed by Thomas et al. [25], which is capable of recovering permutation matrices Z_1 and Z_2 from the known relationship $Y = Z_1 X Z_2$ (where X is a known matrix and Y is the observed obfuscated matrix).

Known X	Observed Y
$W_e W_h$	$\Pi W^{\star}_{\text{embed}} W_h \Pi^T$
$W_e W_{\text{query}} (W_e W_{\text{key}})^T$	$\Pi W^{\star}_{\text{embed}} W_{\text{query}} (\Pi W^{\star}_{\text{embed}} W_{\text{key}})^T$
$W_e W_{\text{gate}}$	$\Pi W^{\star}_{\text{embed}} W_{\text{gate}} \hat{Z}_{\text{ffn}}$
$W_e W_{\text{up}}$	$\Pi W^{\star}_{\text{embed}} W_{\text{up}} \hat{S}_{\text{ffn}} \hat{Z}_{\text{ffn}}$
$W_{\text{down}} W_h$	$\hat{Z}^{-1}_{\text{ffn}} \hat{S}^{-1}_{\text{ffn}} W_{\text{down}} W^{\star}_{\text{head}} \Pi^T$
$W_e W_{\text{router}}$	$\Pi W^{\star}_{\text{embed}} \text{norm}(W_{\text{router}}) \hat{Z}_{\text{router}}$

Table 9: Weight combinations to recover Π with VMA

The core idea of VMA is to first eliminate one permutation matrix through column-wise sorting, then recover the other one via neighbor matching. Specifically, let $\text{RowSort}(\cdot)$ denote the sorting operation along each row. it holds that $\text{RowSort}(Y) = \text{RowSort}(XZ_2)$. By comparing the rows of $\text{RowSort}(XZ_2)$ and $\text{RowSort}(X)$, attackers can recover Z_1. Table 9 summarizes weight combinations in `AloePri` that exhibit a similar structural relationship to $Y = Z_1 X Z_2$. For `AloePri`, weights from multiple decoder layers can be used to conduct VMA. Therefore, we adopt a voting mechanism to utilize VMA results of all layers.

Invariant Attack (IA). Obfuscation mechanisms are vulnerable to this attack if they exhibit *invariants* which remain unchanged during the obfuscation transformation [34]. Based on `AloePri`'s offline model obfuscation process, we present two types of IAs: Gate-IA and Attn-IA.

Gate-IA: This attack leverages W_{gate} to exploit statistical invariants. Let $\boldsymbol{e}$ and $\widetilde{\boldsymbol{e}}$ represent the plaintext and obfuscated embedding vectors, respectively. If no noise is added to the embedding layer ($\alpha_{\text{e}} = 0$), permutation preserves the

statistical property of gate-weighted embeddings, resulting in $\text{Avg}(\boldsymbol{e}W_{\text{gate}}) = \text{Avg}(\widetilde{\boldsymbol{e}}\widetilde{W}_{\text{gate}})$. This average value serves as an invariant: it remains consistent between plaintext and obfuscated data. Attackers can use gate weights from different decoder layers to construct such invariant vectors, thereby reversing the token mapping relationship. To defend against Gate-IA, `AloePri` must add sufficient noise to embeddings by increasing α_{e} so that the statistical consistency of the invariant is disrupted.

Attn-IA: This attack splits W_{query} and W_{key} into blocks (aligned with RoPE block structures) to exploit mathematical invariants. For analysis, we first ignore the head permutation and block permutation techniques introduced in Algorithms 2. Let $W^{(i)}_{\text{query}}$ and $\widetilde{W}^{(i)}_{\text{query}}$ denote the plaintext and obfuscated weights of the i-th attention head, respectively. Define $\boldsymbol{q} = \boldsymbol{e} \cdot W^{(i)}_{\text{query}} = [q_1, \cdots, q_{\frac{d_{head}}{2}}]$, $\widetilde{\boldsymbol{q}} = \widetilde{\boldsymbol{e}} \cdot \widetilde{W}^{(i)}_{\text{query}} = [\widetilde{q}_1, \cdots, \widetilde{q}_{d_{\text{head}}}]$, $Q = W_e W^{(i)}_{\text{query}}$, and $\widetilde{Q} = \widetilde{W}_{\text{embed}}\widetilde{W}^{(i)}_{\text{query}}$. For any block index j, if e and $\widetilde{e}$ correspond to the same token, the mathematical relationship $e(Q^T[:, j]Q[:, j])^{-1}e^T = \widetilde{e}(\widetilde{Q}^T[:, j]\widetilde{Q}[:, j])^{-1}\widetilde{e}^T$ holds as an invariant. Attackers can iterate over all embeddings to verify this invariant relationship, enabling recovery of Π. To defend against Attn-IA, in addition to noise addition, `AloePri` applies permutation over attention heads and blocks to further enhances security by breaking the block-wise consistency required for the invariant to hold.

Internal State Attack (ISA). The attack optimizes input embeddings by leveraging the loss derived from hidden states. Specifically, the attacker first records the hidden states $State_1$ when clients request inference using their private input X_1. Subsequently, attackers randomly initialize X_2 and feeds it into the pretrained model to evaluate $State_2$. The attacker can use $State_1$ and $State_2$ to evaluate the loss, thereby optimizing X_2 to recover X_1. In `AloePri`, model weights are perturbed with noise and transformation, so that all hidden states during the forward computation process are also noisy and transformed. Consequently, the recovered input data through ISA would differ significantly from the original model input, and cannot be directly used to recover private information.

Inversion Model Attack (IMA). With knowledge of the obfuscation mechanism, the attacker can train a model for embedding inversion [26]. During the training process, the attacker iterates over a public training dataset and generates obfuscated embeddings using the target obfuscation mechanism. With these obfuscated embeddings as inputs, the attacker trains the model to generate raw plaintext embeddings or token indices. In the experiments, we train a Qwen2 model with 2 decoder layers and 8 attention heads to invert obfuscated embeddings to plaintext token embeddings.

Token Frequency Leakage. `AloePri` applies the deterministic token obfuscation based on the secret permutation. Therefore, an obfuscated text can be regarded as a substitution cipher and still retains the information about the statistical distribution of token frequencies in the plaintext. To investigate how token-level frequency information impacts privacy, we test the token frequency matching attack (TFMA) and the substitution deciphering attack (SDA) [27]. In both types of attacks, attackers attempt to recover substitution ciphertext based on word frequency information from a prior dataset. In TFMA, the attacker uses the prior dataset to count token frequencies, which are then used to match the frequency of ciphertext tokens in the client's data for token recovery. SDA introduces a recurrence encoding that converts substitution ciphers (1:1 and homophonic) into integer sequences by replacing every unique cipher symbol with an integer symbol standing for its frequency rank. A Transformer-based causal language model is trained on pairs of plain and cipher texts to learn symbol recurrence relations. Then the model is used to translate subsequent ciphertexts to plaintexts.

D.2 Hyperparameter Setting

For all experiments, we fix the generation hyperparameters as follows: temperature = 0.65, top-k = 20, top-p = 0.95, maximum sequence length (max-seq-len) = 8192, and data type (dtype) = bfloat16. Table 10 summarizes the privacy hyperparameters adopted in our experiments, including the noise coefficients for the embedding layer and model head (α_e, α_h), expansion size h, coefficient of the key matrix λ, and block permutation parameters β, γ for attention layers.

Table 10: Privacy hyperparameter settings of experiments.

Experiment	α_e	α_h	λ	h	β	γ
Fig. 3	$\triangle$[1]	$\triangle$	0.3	128	8	$1e^3$
Fig. 4	1.0	0.2	$\triangle$	128	8	$1e^3$
Fig. 5	1.0	0.2	0.3	$\triangle$	8	$1e^3$
Tab. 2, 6, 7	1.0	0.2	0.3	128	8	$1e^3$
Tab. 3	0.5,1.0[2]	0.2	0.3	128	8	$1e^3$
Tab. 4	1.0	0.2	$\triangle$	128	$\triangle$	$\triangle$

[1] $\triangle$ denotes the hyperparameter tuned in the corresponding experiment.
[2] α_e is set to 0.5 for Llama3-8B and 1.0 for all other models.

D.3 Comparison Experiment

Hyperparameter Settings for Baselines. In the experiment presented in Table 2, we tuned the hyperparameters of the baseline methods and `AloePri` to ensure a fair comparison. For `SANTEXT` and `RANTEXT`, we set their privacy budgets to 3 and 30, respectively. For `DP-Forward`, we adopted the LDP mode and set the privacy budget to 5. Additionally, we set the sensitivity to 1. We positioned the noise injection at the hidden states of the 1st decoder layer, thereby offloading the embedding layer and the first decoder layer (accounting for nearly 1B parameters) to the client. Meanwhile, the hidden states were not normalized before noise injection to avoid substantial accuracy loss in the Qwen model.

To reproduce `SGT`, we trained a 1.6B-parameter model following the Qwen2 architecture, which is used to generate the mean and covariance of embedding noise. This model comprises 2 decoder layers, 8 attention heads, a hidden size of 5120, and an intermediate size of 20480 for FFN. We trained the model on 1M text samples from the OpenOrca dataset and determined the training parameters through grid search. Specifically, we set the learning rate to $3e^{-4}$ and trained the model for 2 epochs. We also set the coefficients for accuracy loss, MI loss, absolute cosine loss, and normalization loss to 1.0, $3e^{-5}$, 30, and 1.0, respectively.

Benchmark Settings. In the classification task SST2, we fine-tuned the model on the target task using LoRA [63]. We apply LoRA to attention weights and set $lora_r = 64, lora_alpha = 64$. We set the learning rate to $2e^{-5}$ for 3 training epochs with a sequence length of 128 and batch size of 64. We evaluated the classification task following the recommended application methods of each method. Specifically, for `SANTEXT` and `RANTEXT`, we first obfuscated the training and validation sets using these methods, followed by fine-tuning and inference. For `DP-Forward`, noise was injected to the outputs of decoder layers in both fine-tuning and inference processes. For `SGT` and `AloePri`, we only applied obfuscation during the inference phase.

Generation tasks do not involve fine-tuning or training processes. Thus, in the evaluation of generation tasks, we directly applied obfuscation methods to the inference process.